



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.1: Mobile Edge Computing (MEC)

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Advanced Network Application Paradigms

This module is a deep dive into the next generation of network technologies, structured into four key topics that explore how networks are being transformed to support modern, demanding applications.

Part 1: Mobile Edge Computing (MEC) (6/7 hours)

- This topic explores **Mobile Edge Computing (MEC)**, a distributed paradigm that brings cloud capabilities to the network's edge. We'll see how MEC drastically reduces latency by processing data locally, enabling real-time applications like **AR**, **VR**, and **video analytics** that are impossible with traditional centralized clouds.

Part 2: Network Slicing (6/7 hours)

- This topic explores **Network Slicing**, a key 5G technology that partitions a single physical network into multiple virtual "slices." Each slice can be customized for specific performance requirements like bandwidth and latency, serving applications from **IoT devices** to **autonomous vehicles**.

Part 3: Multimedia Networking (6/7 hours)

- This topic is dedicated to the challenges of delivering high-quality multimedia content. We'll cover protocols and techniques for ensuring quality of service (QoS) and managing jitter and latency. Key topics include **adaptive bitrate streaming**, **WebRTC**, and the role of **CDNs**.

Part 4: Distributed Machine Learning (DML) (6/7 hours)

- This topic explores **Distributed Machine Learning (DML)**, where ML models are trained and run across a network of devices. It enables local computation, reducing reliance on a central server. We'll discuss **federated learning**, where a model is trained on devices without sharing raw data, and its benefits for **personalized assistants** and **IoT analytics**.

Agenda for Part 1: MEC

Foundations & Motivation: The Physics, Economics, and 5G Architecture Driving MEC.

The ETSI MEC Architecture: A Deep Dive into the ETSI ISG MEC Specifications and APIs.

Computational Offloading: Formal Modeling, Optimization Algorithms, and Performance Analysis.
ottimizzo fra la scelta se conviene o meno spostare la comp.

Use Cases & The Ecosystem: Technical Challenges and Implementation of Key MEC Applications.

Foundations & Motivation



The Convergence of Cloud, Telecom, and Computing

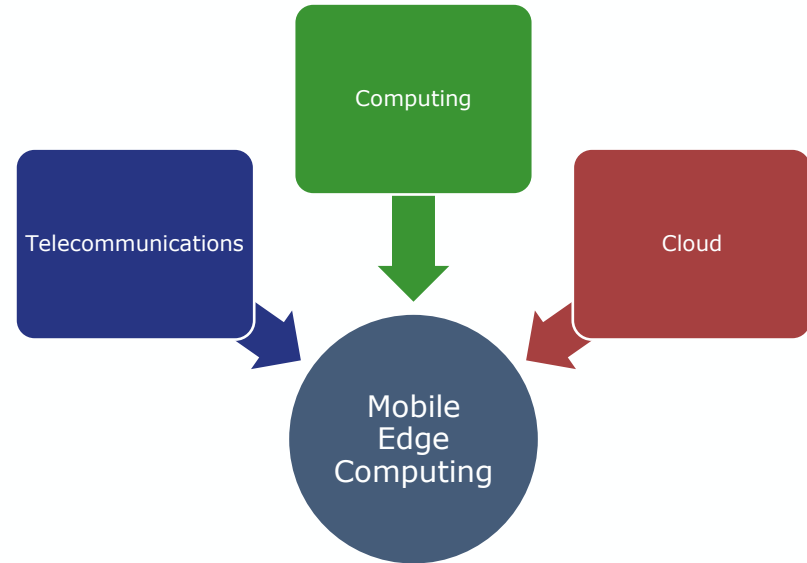
tramite architettura mec ci si astrae => si estende il concetto ad applicazioni in senso lato (anche a livello utente)

MEC isn't just an incremental improvement; it's a revolutionary paradigm born from the intersection of three massive industries:

infrastruttura
simil cloud
tanti piccoli
elementi cloud
ma vicini

- **Cloud:** Contributes the "as-a-service" model, virtualization/containerization, and a vast ecosystem of developers and applications.
- **Telecommunications:** Provides ubiquitous mobile connectivity, real-time network information, and physical proximity to billions of end-users.
- **Computing:** Provides the processing capabilities for executing low-latency workloads at the edge.

a differenza del cloud, si cerca di avere casi di low-latency



The Physics of Latency: Deconstructing Mobile Network RTT

nel caso ci sia una richiesta di dati

The **Round-Trip Time (RTT)** is not a single number; it's a sum of serial delays. Let's analyze the path from a mobile device (UE) to a central cloud server.

- **UE Processing Delay** (T_{ue}): $\sim 1\text{-}2$ ms ritardo per formare il dato da inviare
- **Air Interface Delay** (T_{air}): Radio frame scheduling, HARQ retransmissions. Variable, $\sim 2\text{-}10$ ms. ritardo necessario perché il pacchetto parta => tempo di interfacciamento con l'altro punto
- **RAN Processing Delay** (T_{ran}): At the gNodeB (base station). $\sim 1\text{-}3$ ms.
- **Transport & Backhaul Delay** ($T_{transport}$): Fiber/microwave link from cell site to the core network. **Variable by distance**, $\sim 5\text{-}20$ ms.
- **Core Network Processing** (T_{core}): At the UPF and other gateways. $\sim 1\text{-}3$ ms.
- **Internet Transit Delay** ($T_{internet}$): **Propagation delay to the cloud data center. $\sim 10\text{-}150\text{+}$ ms.** trasmissione tramite la rete internet

MEC's Impact: A MEC host is co-located with the RAN. By breaking traffic out at the base station, MEC **eliminates** $T_{transport}$, T_{core} , **and** $T_{internet}$ from the RTT, which are the largest and most variable components. interrompe la catena di trasmissione a livello di RAN

What is MEC? The Formal Definition

The **European Telecommunications Standards Institute (ETSI)**, which standardizes MEC, defines it as:

"Mobile Edge Computing provides an IT service environment and cloud-computing capabilities at the edge of the mobile network, within the Radio Access Network (RAN), and in close proximity to mobile subscribers."

primo segmento della nostra rete

Deconstructing the Definition:

- **"IT service environment"**: It's designed to run standard applications using common tools, **not proprietary telecom functions.**
- **"Cloud-computing capabilities"**: **It is elastic, virtualized, multi-tenant, and programmable via APIs.**
- **"Edge of the mobile network"**: This is the crucial part. It's not just any edge; it's the **network operator's edge.** diventa un elemento interno all'infrastruttura di rete => ho tutta una serie di info nuove da poter utilizzare
- **"Within the RAN"**: It is physically co-located with the mobile network's infrastructure, like 4G/5G base stations.

MEC vs. Fog: A Matter of Scope and Integration

non solo integrato all'interno della ran ma ancora + distribuito

Fog Computing: A broader, more conceptual term for a distributed computing layer that exists anywhere between the cloud and the end devices. It can include on-premise routers, industrial gateways, or any local compute resource.

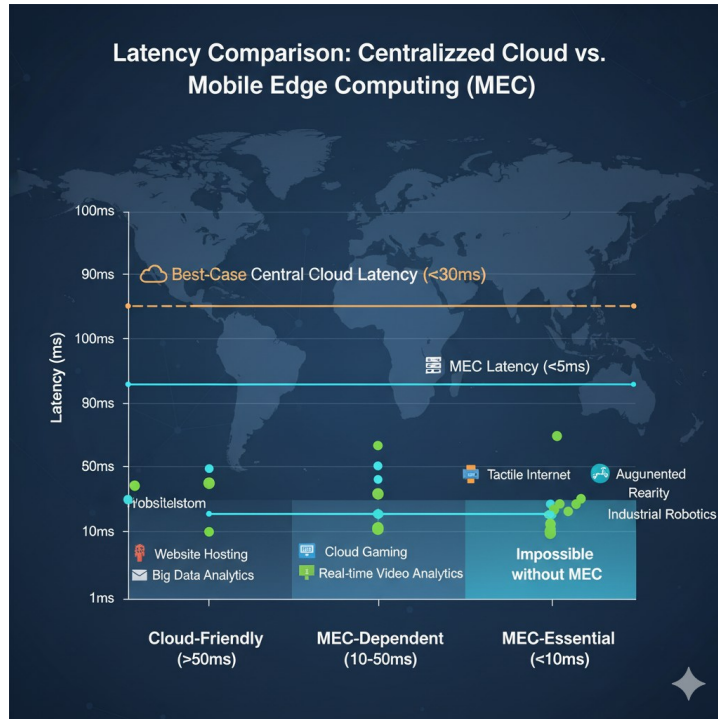
Mobile Edge Computing (MEC): A specific, standardized implementation of Fog Computing that is deeply integrated with the mobile network.

The Key Difference: MEC has privileged, standardized access to mobile network functions and information (like real-time bandwidth or cell load) that a generic Fog node typically does not. MEC can directly influence mobile traffic routing.

MEC vs. Cloud vs. Fog: A Technical Comparison

Feature	Centralized Cloud	Fog Computing	Mobile Edge Cloud (MEC)
Location	Centralized Hyperscale DC	Heterogeneous, opportunistic	Standardized, at the Mobile RAN
Latency	High (>30ms), variable	Low to Medium, variable	Ultra-low (<10ms), predictable
Network Context	None (over-the-top)	Limited (e.g., LAN info)	Rich (3GPP RAN info via APIs)
Mobility Support	Client-side only	Ad-hoc, non-standardized	Standardized (via 3GPP integration)
Key Standard		IEEE 1934	ETSI ISG MEC, 3GPP
Primary Enabler	Economies of Scale	IoT Gateways, local servers	5G Architecture (CUPS, UPF)

Why MEC? Driver 1: The Latency Imperative



Different applications have vastly different tolerances for delay. The need for ultra-low latency is a primary driver for MEC.

- Cloud-Friendly Zone (>50ms): Website hosting, Big Data analytics, Email.
- MEC-Dependent Zone (10-50ms): Cloud Gaming, Real-time video analytics.
- MEC-Essential Zone (<10ms): Tactile Internet, Augmented Reality, Autonomous Vehicles, Industrial Robotics.

si copre un buco che il cloud non riusciva a risolvere
=> sotto i 10 ms serve usare il MEC!

Why MEC? Driver 2: The RAN Bottleneck

The Radio Access Network (RAN) is the most constrained and expensive part of the mobile network. MEC is critical for optimizing its use.

rete RAN costituisce collo di bottiglia => grandi quantità di dati da trasportare lungo la mia rete

A Concrete Example: Video Surveillance

- A single 4K security camera streams at approximately **15 Mbps**.
- A smart factory deploying 100 of these cameras would generate **1.5 Gbps** of continuous uplink traffic.
- Sending this massive stream over the mobile network's backhaul to a central cloud 24/7 is economically and technically unfeasible.

The MEC Solution: By processing the video at a local MEC host, the system only needs to send small alerts or metadata (e.g., "person detected in restricted area") over the backhaul. **This can reduce the required bandwidth by over 99%.**

evito che tutti i dati grezzi vengono condivisi al punto centrale => si riduce drasticamente la quantità di dati

Why MEC? Driver 3: Network & Location Context

A MEC application is unique because it is "network-aware." Through standardized APIs provided by the MEC platform, an application can ask the network critical questions in real-time, such as:

- "What is the current estimated throughput for user X?"
- "Which cell ID is user Y currently connected to?"
- "What is the precise location of device Z within the cell?"

This enables a new class of **context-aware applications** that can intelligently adapt their behavior based on the user's real-time network conditions and physical location.

Why MEC? Driver 4: Data Privacy & Sovereignty

i dati rimangono dentro al sistema del mio operatore di rete

Sending all data to a centralized public cloud is not always desirable or permissible.

Privacy: Processing sensitive, personal data locally at the edge can prevent it from ever traversing the public internet. This is critical for applications in healthcare (patient data), home security (camera feeds), and enterprise settings (proprietary data).

Data Sovereignty: Many countries have strict laws (like the EU's GDPR) that mandate their citizens' data must be processed and stored within national borders. MEC provides a powerful platform for telco operators to offer services that guarantee in-country data processing, ensuring legal compliance.

MEC in the 5G Ecosystem

While MEC was retrofitted onto 4G networks, the 5G system was designed from the ground up with **MEC as a core**, native principle.

Several **key features of the 5G architecture are specifically designed to enable and enhance MEC:**

- **URLLC (Ultra-Reliable Low-Latency Communication):** The 5G air interface provides the high-performance radio link that MEC applications demand.
- **CUPS (Control and User Plane Separation):** The most important architectural enabler for distributing the network to the edge. *installo particolari funzioni di rete nei nodi edge => aggiungo funzioni del piano utente!*
- **NEF (Network Exposure Function):** The standard 5G component for securely exposing network information and capabilities to applications, including those at the edge.

requisiti posti nel 5G => MEC cerca di implementarli

5G Enabler: Control and User Plane Separation (CUPS)

This is a fundamental architectural shift from 4G, and it's a critical concept for **CSN** students.

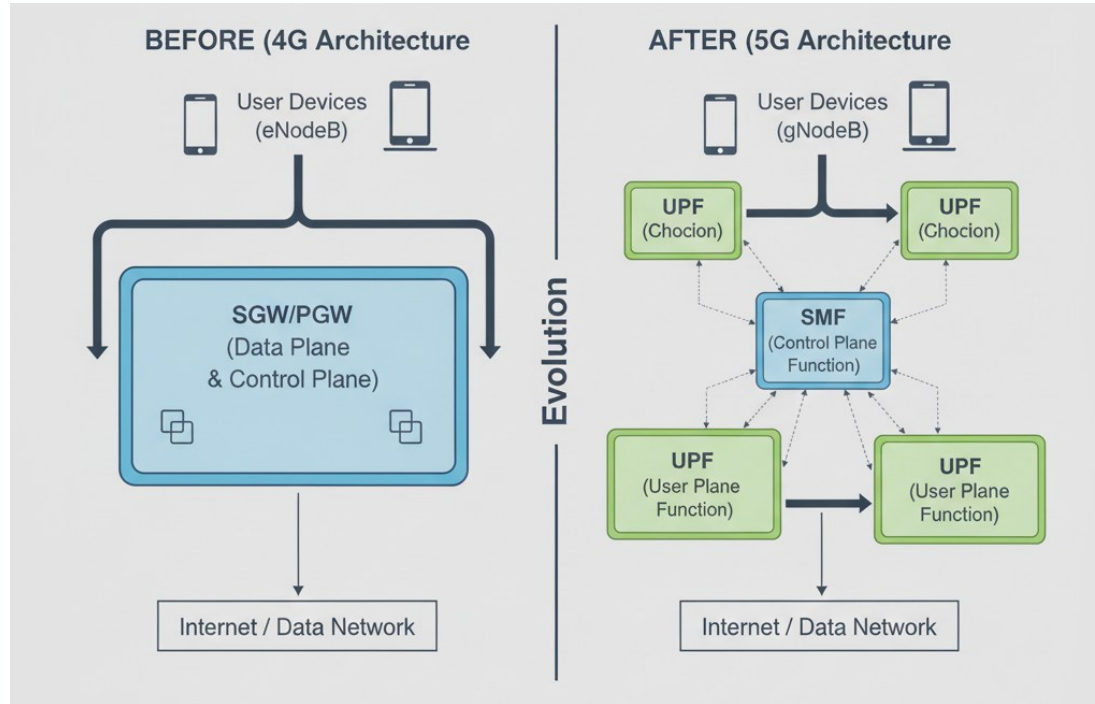
In 4G: The gateway that handled both user data (User Plane) and session management (Control Plane) was a monolithic, centralized hardware box.

In 5G: These two roles are separated.

- **SMF (Session Management Function):** The control plane "brain" that manages user sessions. It can remain centralized.
- **UPF (User Plane Function):** The data plane "forwarder" that actually routes user packets.

The Revolution: This separation allows the control plane to be centralized while many small, lightweight UPFs can be distributed and deployed at the network edge, right alongside the MEC hosts.

5G Enabler: Control & User Plane Separation (CUPS)



The UPF: The Gateway to the Edge

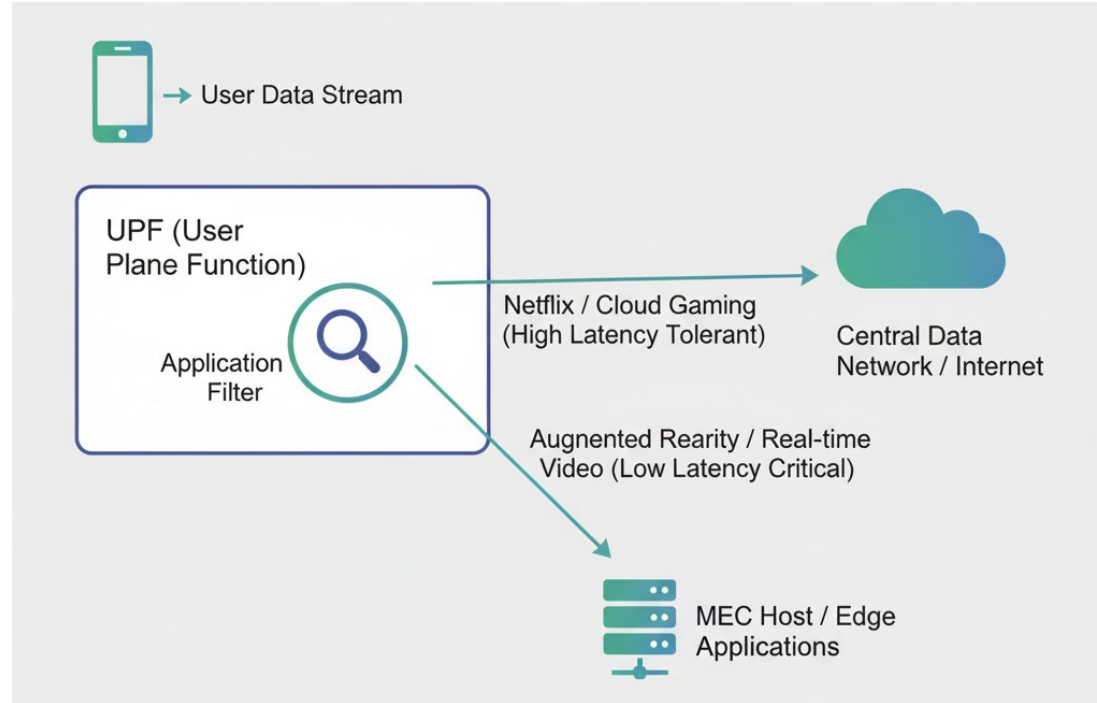
The distributed UPF at the edge acts as an intelligent traffic cop for user data.

The central 5G Core (specifically the SMF) can establish a user session and send specific traffic steering rules to the UPF.

Example Rule: "For this user's session (PDU Session), if the traffic is identified as 'low-latency AR gaming' (based on an Application Filter), **then break it out locally and forward it directly to the co-located MEC host at IP address X.** All other traffic should be forwarded to the central data network." [interagisco direttamente con il piano utente](#)

This capability, called **local traffic breakout**, is the fundamental mechanism that enables MEC services to intercept and process user traffic at the edge.

The UPF: The Gateway to the Edge



The ETSI MEC Architecture



Why We Need a Standard Architecture

Without a standard, every vendor would build their own proprietary edge solution. This would lead to a fragmented ecosystem.

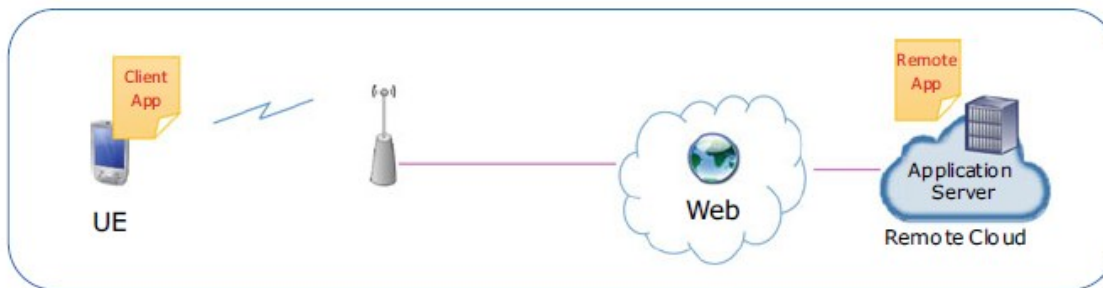
Vendor Lock-in: Application developers would have to write their apps specifically for one operator's edge platform, and it wouldn't work on another's.

Lack of Interoperability: Mobile operators wouldn't be able to mix and match the best platform components from different vendors.

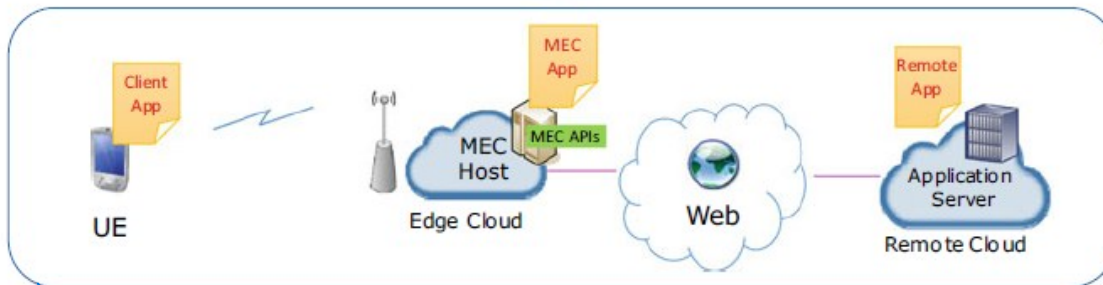
The **European Telecommunications Standards Institute (ETSI)** created the **Industry Specification Group for Mobile Edge Computing (ISG MEC)** to solve this problem by defining a single, open, and interoperable framework.

Reference scenario for MEC

scenario tradizionale:
app utente che vedono
web semplicemente
come un tubo per
il trasporto dati



qui invece si vanno
a replicare i servizi del
cloud ma anche di dare
la possibilità di interagire
con la rete tramite
interfacce api e
quindi implementare
nuovi servizi (sfruttando
le info che la rete ci fornisce)



ETSI MEC Specifications: The Key Documents

The architecture is not just a concept; it's defined by a series of detailed technical specifications (TS) and group specifications (GS). It's important to know the key documents.

- **MEC 003**: Framework and Reference Architecture. The foundational document.
- **MEC 011**: Mp1 Application Enablement API. Defines the critical RESTful API between applications and the platform. We will examine this in detail.
- **MEC 012**: **Radio Network Information API**. A specific part of MEC 011 that defines the RNIS.
- **MEC 016**: **V2X Support API**. Shows how MEC can be a platform for connected **car** applications.
- **MEC 010**: MEC Platform and Host Management. Defines the management interfaces.

se un interfaccia mi fornisce particolari info => tipo livello di congestione
=> posso sviluppare una particolare applicazione che sfrutta quel dato/info del sistema
seguendo un particolare standard/api
=> applicazioni proprio a livello utente (livello 7) e non funzioni di rete (NFV) => tipo routing (livello trasporto)

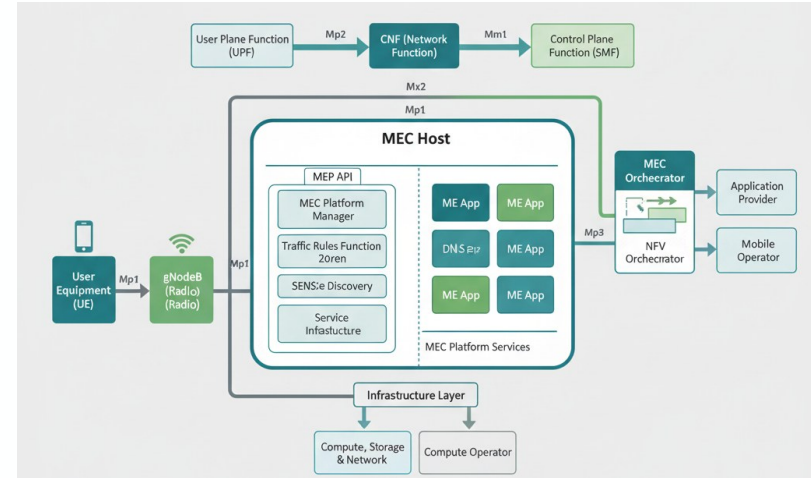
The Standard Blueprint: ETSI MEC Framework

This is the high-level functional architecture. It defines the key components and the reference points (interfaces) between them.

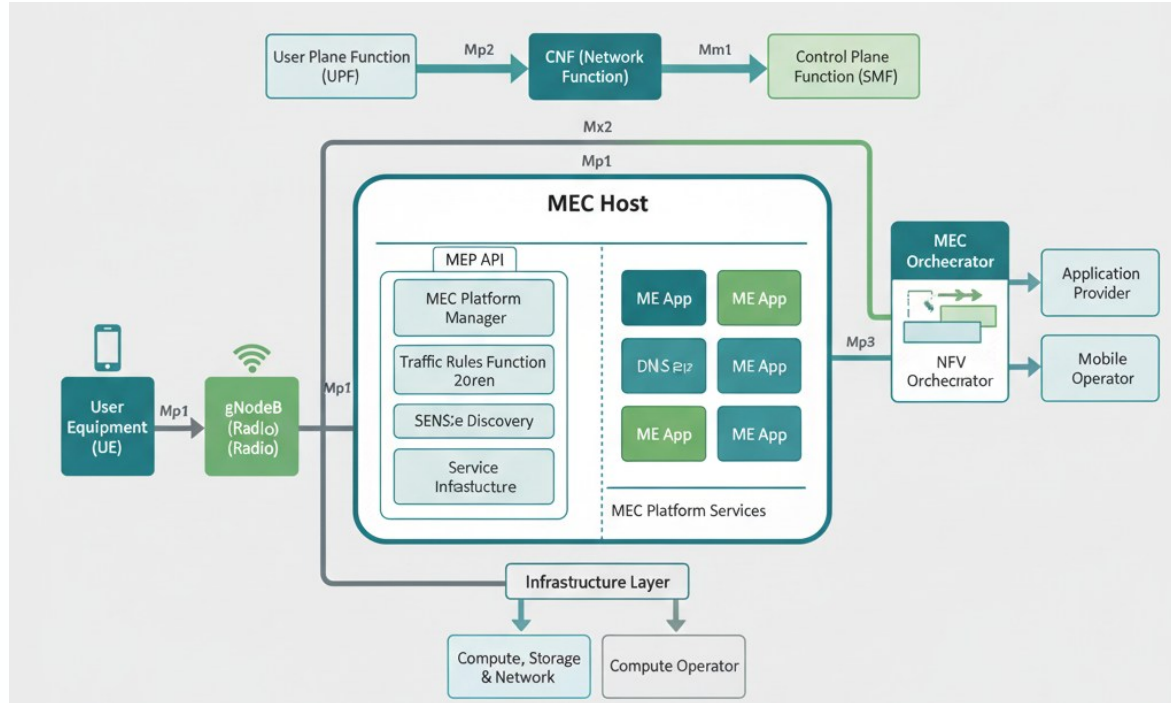
The architecture logically separates the system into:

- **MEC Host Level:** The physical location and resources.
- **MEC System Level:** The overall management and orchestration of multiple hosts.

This standardized model is the foundation for building an open and interoperable edge ecosystem.



The Standard Blueprint: ETSI MEC Framework



Core Principles of the MEC Architecture

The ETSI framework is built on several key principles:

ogni vendor può creare i propri componenti
purché rispettino le interfacce

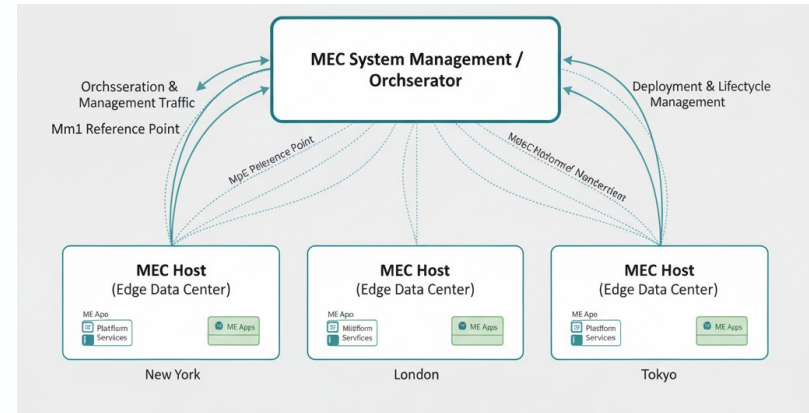
- **Openness:** The interfaces (reference points) are open and standardized, allowing components from different vendors to interoperate.
- **Multi-tenancy:** A single MEC host must be able to securely host applications from multiple different application providers (tenants).
- **Virtualization:** It leverages the principles of NFV, assuming that all applications and platform services run on a virtualized infrastructure (the NFVI).
- **Network Integration:** The architecture is not an overlay; it is deeply integrated with the 3GPP mobile network (4G/5G) to gain access to network information and influence traffic routing.

elemento aggiuntivo d'intelligenza rispetto alla rete 5G

System View vs. Host View

It's important to differentiate between the two main levels of the architecture.

- **MEC Host:** The physical and logical entity at a **single edge location** (e.g., a single cell tower). It consists of the hardware, the virtualization layer, the MEC platform, and the applications running on it.
- **MEC System:** The complete collection of all MEC Hosts and the centralized management functions that oversee the entire distributed system. An operator might have thousands of MEC Hosts in their MEC System.



The MEC Host: **Where the Edge Lives**

The MEC Host is the complete set of hardware and software resources at a specific edge deployment site. [tutte risorse hard/software di un particolare elemento edge](#)

It comprises two main parts:

- **MEC Platform:** [The software environment](#) that provides services to applications.
- **NFV Infrastructure (NFVI):** [The underlying hardware and virtualization](#) layer that provides the raw compute, storage, and networking resources.

The physical constraints of an edge location (power, cooling, space) heavily influence the size and capability of a MEC Host.

[si deve tener conto di tutta una serie di caratteristiche fisiche simil-cloud](#)

MEC Host Hardware: COTS Servers & Specialized Accelerators

The foundation is **Commercial Off-The-Shelf (COTS) x86 servers**, just like in NFV.

However, MEC workloads are often highly specialized and require hardware acceleration to achieve the necessary performance.

Common Accelerators found in MEC Hosts:

- **GPUs (Graphics Processing Units)**: Essential for video rendering (AR/VR, Cloud Gaming) and parallel processing for AI inference.
- **FPGAs (Field-Programmable Gate Arrays)**: Can be programmed to accelerate very specific networking or application tasks with high efficiency.
- **AI Accelerators**: Specialized ASICs (like Google's TPUs) designed for high-performance, low-power AI model inference.

Hardware Acceleration: GPU vs. FPGA

The choice of accelerator depends on the workload.

GPU (Graphics Processing Unit):

- **Architecture:** Massively parallel Single Instruction, Multiple Data (SIMD) processors.
- **Best for:** Highly parallelizable tasks like ML model inference (tensor math) and video encoding/rendering.
- **Pros:** Easy to program (using frameworks like CUDA, TensorFlow). Rich software ecosystem.
- **Cons:** Higher power consumption, less flexible for non-parallel tasks.

FPGA (Field-Programmable Gate Array):

- **Architecture:** A sea of reconfigurable logic blocks and interconnects. Can be programmed to create custom hardware circuits.
- **Best for:** Ultra-low latency, deterministic packet processing, protocol acceleration (e.g., implementing parts of the 5G radio stack), custom cryptography.
- **Pros:** Very low latency, highly power-efficient for specific tasks.
- **Cons:** Difficult to program (requires hardware description languages like Verilog/VHDL), less flexible than a GPU.

MEC Host Virtualization: The Runtime Environment

esattamente come NFV

The virtualization layer abstracts the physical hardware. The MEC architecture supports two main models:

Hypervisor-based (**Virtual Machines**):

- Each ME App runs in its own full Guest OS, providing strong isolation.
- This was the initial approach, leveraging existing NFV infrastructure.

Container-based: molto + portabile

- Each ME App runs as a lightweight container, sharing the Host OS kernel.
- This is the **dominant modern approach** for MEC because it offers a much lower resource footprint and faster startup times, which are critical for the dynamic edge environment. The orchestrator is typically Kubernetes.

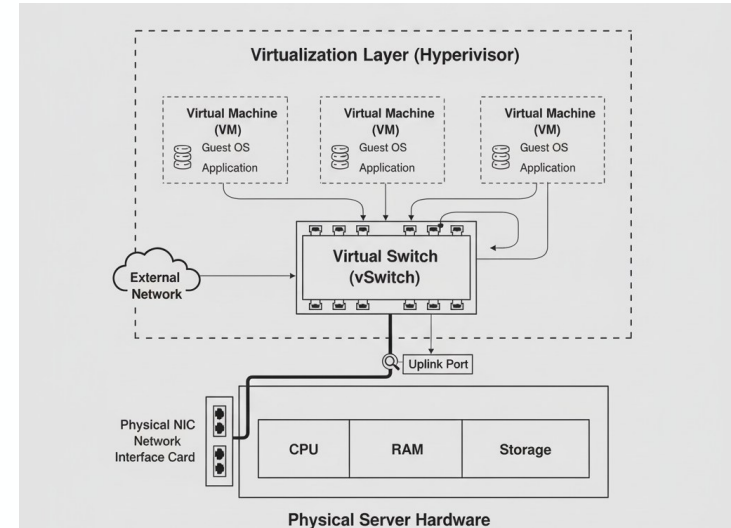
MEC Host Networking: The Role of the Virtual Switch

permette di gestire in maniera efficiente come gestire il traffico

The vSwitch (e.g., Open vSwitch) is the software component responsible for all networking inside the MEC host.

Key Responsibilities:

- **Intra-Host Communication:** Connecting ME Apps (containers/VMs) running on the same server.
- **External Communication:** Connecting ME Apps to the physical NICs to communicate with the mobile network and the internet.
- **Traffic Steering:** The vSwitch is the enforcement point for the Traffic Offload Function (TOF), directing user plane traffic towards the correct ME App.



MEC Networking: Achieving High Performance

For demanding networking applications (like a virtual router or a real-time video processor), the standard kernel networking stack and **vSwitch can be a bottleneck**.

To achieve line-rate performance, MEC hosts leverage the **same high-performance I/O technologies used in NFV**:

- **SR-IOV (Single Root I/O Virtualization)**: Allows an ME App (VM/container) to **bypass the vSwitch** and get direct, exclusive access to a "virtual slice" of the physical network card. This provides the lowest possible latency.
- **DPDK (Data Plane Development Kit)**: A set of libraries that allows an ME App to process packets directly in user space, **bypassing the slow kernel network stack**. This provides the highest possible throughput (packets per second).

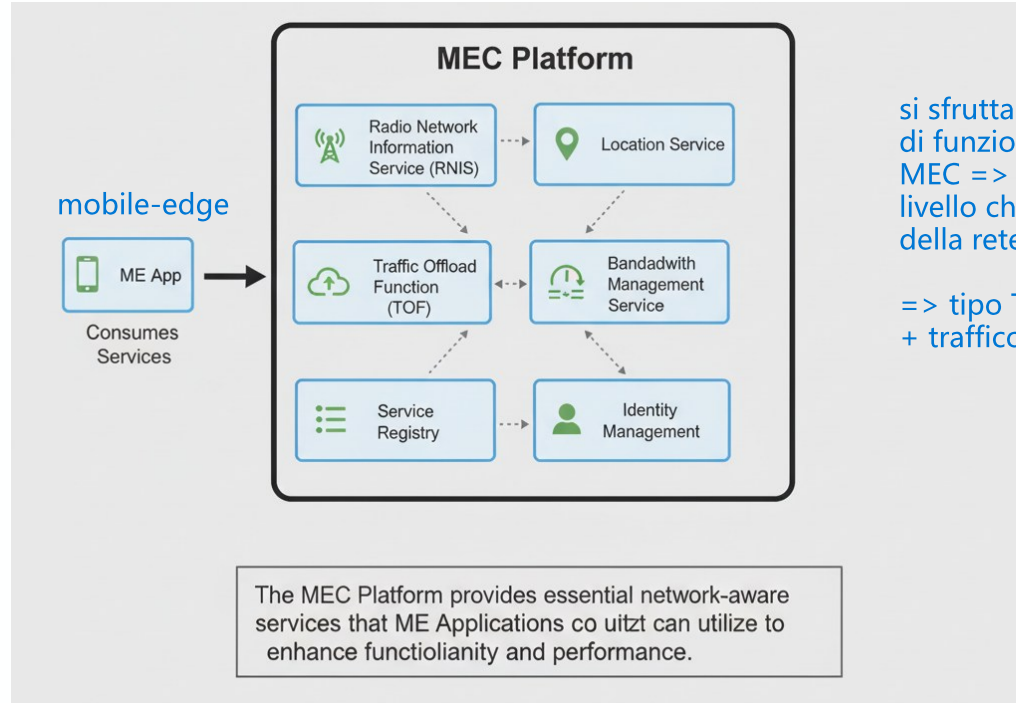
The MEC Platform: The Edge Service Environment

If the MEC Host is the Infrastructure (IaaS), then the **MEC Platform** is the **Platform as a Service (PaaS)**.
allo stesso modo del cloud => servizi esposti alle API per creare le funzioni da usare all'interno della rete => sfruttando la conoscenza dello stato di questa

It is a collection of essential software services that are exposed to applications via APIs.

Its Goal: To abstract the complexity of the underlying mobile network and infrastructure, allowing developers to easily build powerful, context-aware applications without needing to be telecom experts.

The Core Services of the MEC Platform



si sfruttano una serie di funzioni fornite dalla piattaforma MEC => elementi di più alto livello che ci danno info sullo stato della rete

=> tipo TOF => si guarda dove c'è + traffico di dati

Platform Service: Radio Network Information Service (RNIS)

The RNIS is one of the most unique and powerful services in MEC.

It collects real-time information from the underlying Radio Access Network (RAN) and exposes it to applications via a standardized API (over the Mp1 reference point).

Information an App can get from RNIS:

- Current Cell ID and signal strength for a user.
- Real-time measurements of radio channel quality.
- Estimated available bandwidth on the uplink and downlink.
- Cell-level congestion information.

tutte info a livello di
cella per un particolare utente
es: se un segnale di un utente è
debole oppure non ha
abbastanza capacità di canale
=> cerco di ridurre la banda di
trasmissione

Example: Using RNIS to Make an App Smarter

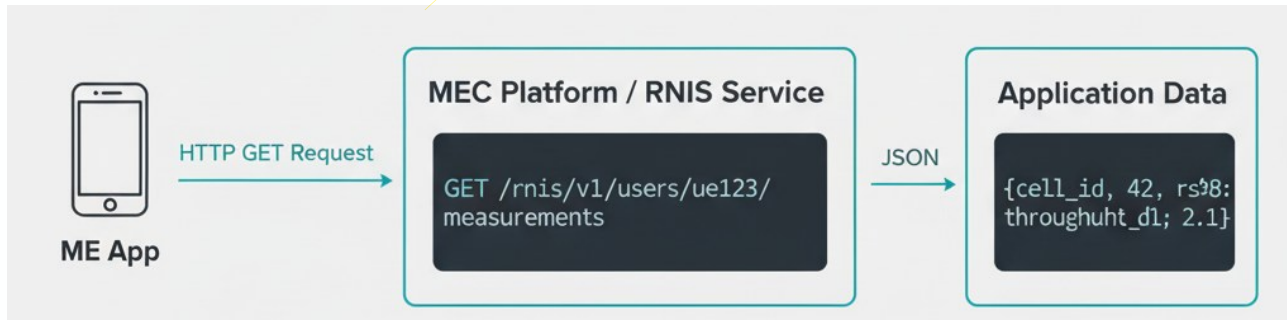
An adaptive video streaming application can use the RNIS to be proactive instead of reactive.

Without MEC: The video player adapts the quality *after* it detects network congestion (leading to buffering).

With MEC:

- The ME App subscribes to the RNIS for bandwidth updates **for a user.**
- RNIS sends a notification: "Warning, cell congestion is increasing. Estimated throughput for this user will drop to 2 Mbps in the next 5 seconds."
- The ME App **proactively** tells the video player to switch **to a lower bitrate *before* any packet loss occurs.**

ho le info di tutti gli utenti! (cosa non possibile nel cloud)



Platform Service: Traffic Offload Function (TOF)

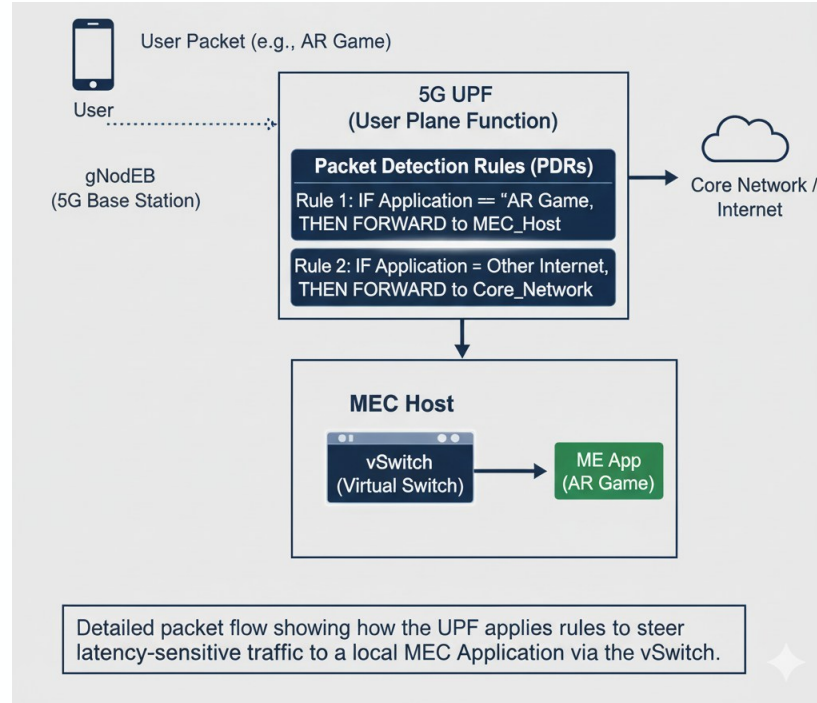
The Traffic Offload Function (or its equivalent integration with the 5G UPF) is the service responsible **for routing user traffic to the ME Apps.**

The MEC Platform Manager configures the TOF with rules that define which traffic flows should be "broken out" locally.

Rules can be based on:

- The application type (e.g., all traffic for "Awesome AR Game").
- The destination IP/domain (e.g., all traffic to local-game-server.mec).
- The user's subscription profile.

TOF: A Packet Flow Diagram



in questo modo riesco
a partizionare il traffico in
funzione dei diversi requisiti

Platform Service: Location Service

The MEC Location Service provides ME Apps with **fine-grained, real-time location information for users and devices.**

It can fuse information from multiple sources to achieve much higher accuracy than GPS alone:

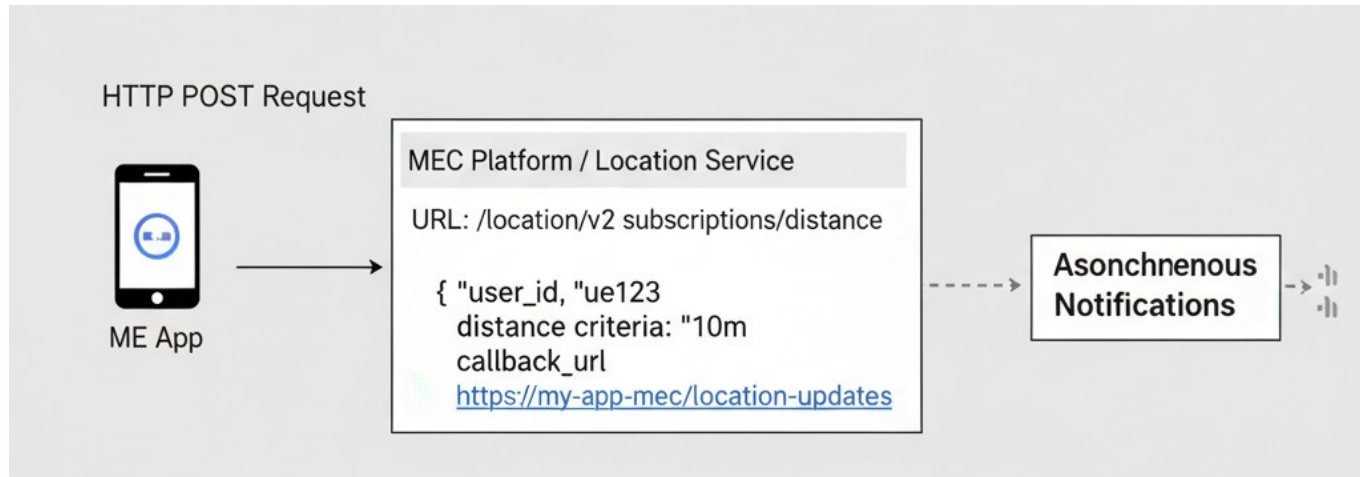
- GPS coordinates from the device.
- **RAN-based information:** Cell ID, timing advance, and other radio measurements that can pinpoint a device's location relative to the cell towers.

This enables location-based services like local advertising, AR navigation, and asset tracking.

se l'utente si trova in una particolare posizione si attiva (o meno) un particolare servizio MEC

Example: Subscribing to Location Updates

A "find-my-friends" type of application running at the edge **could use the Location Service.** *tipo in situazioni di emergenza locale => avverto utenti che si trova in quell'area*



Platform Service: Bandwidth Management Service

The radio network is a shared, and often congested, resource. Not all applications are created equal; a real-time AR application is more important than a background file download.

The **Bandwidth Management Service** allows authorized ME Apps to request specific Quality of Service (QoS) treatment for their traffic flows.

An App can request:

- A **guaranteed minimum bitrate**. | caso di applicazioni real-time o app che richiedono particolari latenze/jitter
- A certain **priority level** for its traffic.
- Specific **latency and jitter** characteristics.

The MEC platform then communicates these requirements to the 5G Core (PCF/SMF), which enforces them in the RAN. This ensures critical applications get the network performance they need to function correctly.

Platform Service: Service Registry

A MEC Host can run many different applications and services from different providers. How do they find and communicate with each other?

The **Service Registry** acts as a dynamic discovery mechanism.

How it works:

- When a new ME App or service starts, it **registers** itself with the Service Registry, advertising its name, location (IP/port), and capabilities.
- Another app that needs to use this service can **query** the registry (e.g., "Find me an available low-latency video transcoding service").
- The registry returns a list of available service endpoints.

This enables a flexible, microservices-based architecture at the edge, where applications can be composed of multiple independent services.

[catena di microservizi](#)

The MEC Platform Manager

gestore della piattaforma del sistema MEC

The **MEC Platform Manager** is the central component within the MEC Platform itself.

Its role:

- Manages the lifecycle of all the other platform services (RNIS, Location, etc.).
- Handles application authentication and authorization. It decides which apps are allowed to use which platform services.
- Maintains the service registry.
- Interfaces with the system-level MEC Orchestrator to manage the lifecycle of ME Apps.

What Makes an Application "Edge-Native"?

cioè non è strettamente necessario sempre avere applicazioni di tipo MEC

Not every application benefits from running at the edge. A "good" ME App is one that is specifically designed to leverage the unique capabilities of the MEC environment.

Key Characteristics:

- **Latency-Sensitive:** The app's performance or user experience is critically dependent on ultra-low latency.
- **High Bandwidth (Uplink):** The app processes large amounts of data generated by the user or local sensors (e.g., video analytics). piuttosto meglio avvicinare e suddividere i dati
- **Context-Aware:** The app's logic can be enhanced by using real-time radio network or location information.
- **Local Focus:** The application primarily serves users within a limited geographical area.

The Lifecycle of a Mobile Edge Application

ho la necessità di capire quando un applicazione ha bisogno o meno di essere fatta partire => ciclo di vita dell'app

Similar to the VNF lifecycle, ME Apps are managed from birth to death by the MEC system.

1. **Onboarding:** The application package (e.g., a container image and its descriptor file) is uploaded to the MEC Orchestrator's catalog.
2. **Instantiation:** An operator or an automated trigger decides to launch the application. The orchestrator selects a suitable MEC host and commands it to create an instance of the app.
3. **Runtime:** The app is running. It can be scaled, updated, or migrated to another host if the user moves.
4. **Termination:** The application instance is shut down, and all its resources are released.

ME App Packaging: VNFs and CNFs

The application software needs to be packaged in a standardized way so the orchestrator knows how to deploy it. The MEC architecture supports two main models, following the NFV evolution:

VNF (Virtual Machine):

- The app is packaged as a complete VM image (e.g., .qcow2, .vmdk).
- **Pros:** Strong isolation. **Cons:** Large, slow to start, high overhead.

CNF (Container):

- The app is packaged as a lightweight container image (e.g., a Docker image).
- **Pros:** Small, starts in seconds, highly efficient. **This is the preferred and dominant method for modern ME Apps.** The application descriptor would be a Kubernetes Helm Chart or Operator.

App State Management at the Edge

A user is connected to a MEC Host at Cell Site A. The ME App instance they are using has built up important "state" about their session (e.g., their character's inventory in a game).

The Problem: What happens when the user moves and connects to Cell Site B? A new application instance might be spun up on the MEC Host at Site B, but it doesn't have the user's session state.

The Solution: This is a complex distributed systems problem. Solutions involve:

- **State Replication:** Proactively copying the state to nearby MEC hosts.
- **A Tiered Architecture:** Storing the critical state in a slightly more centralized (but still low-latency) regional data center that multiple MEC hosts can access.

The MEC Orchestrator (MEAO)

The **MEC Orchestrator (MEAO)** is the centralized management component that has a global view of the entire MEC System.

It oversees all the distributed MEC Hosts.

Key Responsibilities:

- Maintains a catalog of all available ME Apps and their descriptors.
- Has an inventory of all MEC Hosts and their available resources.
- Makes high-level decisions about application placement and lifecycle management.

MEAO vs. NFVO: The Relationship

In the ETSI framework, these are distinct logical roles, but in practice, they are often combined.

- **NFV Orchestrator (NFVO)**: Is a generic orchestrator for any type of VNF/CNF (e.g., a virtual mobile core). It is resource-focused.
- **MEC Orchestrator (MEAO)**: Is a specialized orchestrator that understands the unique requirements of edge applications. It is service-focused and topology-aware.

For example, the MEAO knows which MEC Host is physically located at a specific cell site, and it uses this information to make placement decisions. An NFVO may not have this level of physical awareness.

MEAO Role: Application Lifecycle Management

The MEAO is the primary entry point for managing applications.

An operator interacts with the MEAO to:

- **Onboard** new application packages.
- **Instantiate** an application (e.g., "Deploy the 'AR-Dinosaur-App' to all MEC hosts in downtown Florence").
- Set up high-level policies for scaling and high availability.
- **Terminate** an application instance.

MEAO Role: VNF/CNF Placement

This is one of the most important and complex functions of the MEAO. When a request to instantiate an application for a specific user arrives, the MEAO must decide **which MEC Host to deploy it on.**

Factors in the Placement Decision:

- The user's current radio cell location.
- The available compute/storage/GPU resources on nearby MEC Hosts.
- The specific latency requirements of the application.
- The cost of deploying on different hosts.
- Any data sovereignty or policy constraints.

This is a sophisticated, multi-variable optimization problem.

MEAO Placement as an Optimization Problem

The MEAO's decision of where to instantiate an application is a classic **NP-hard optimization problem**, often modeled as a variation of **Bin Packing** or a **Generalized Assignment Problem**.

Objective Function:

- *minimize* $\Sigma(\text{latency}_i)$ or *maximize* $\Sigma(\text{revenue}_i)$
 - Minimize the total latency for all users, or maximize the total revenue

Given:

- A set of MEC Hosts H , each with capacity constraints (C_h for CPU, M_h for memory, G_h for GPU).
- A set of Applications A to be placed, each with resource requirements (c_a, m_a, g_a).
- A matrix of latencies L_{ah} between the user requesting app a and host h .

Constraints:

- The sum of resources of all apps placed on a host cannot exceed its capacity.
- Each app must be placed on exactly one host.

Solution: Because this is NP-hard, real-world orchestrators use **heuristics** or **approximation algorithms** to find a "good enough" solution quickly.

MEAO Role: Service Continuity during Mobility

The MEAO is responsible for ensuring a seamless user experience as users move.

The Process:

1. The mobile network's control plane (e.g., the AMF in 5G) notifies the MEAO that a user is about to perform a **handover** from Cell A to Cell B.
2. The MEAO checks if the ME App the user is consuming is available on the MEC Host at Cell B.
3. If not, the MEAO may trigger the instantiation of a new instance on the target host.
4. It also orchestrates the transfer of the application state (as discussed before) so that the new instance can seamlessly take over the session.

The Importance of Standard Interfaces

The ETSI architecture defines **a set of standardized interfaces**, called **Reference Points**, between the major functional blocks.

Why are they important? They allow an operator to **"mix and match"** components from different vendors. For example, they can use a MEC Platform from Vendor A with a MEC Orchestrator from Vendor B, and as long as both correctly implement the standard interfaces, they will work together.

Mp1: The Application-to-Platform Interface

The **Mp1** reference point is the API exposed by the **MEC Platform** to the **Mobile Edge Applications**.

This is how an application consumes all the value-added services from the platform.

Functions over Mp1:

- Discovering available platform services.
- Consuming services like **RNIS and Location**.
- Requesting traffic rules to be set up.

ETSI defines this as a **RESTful API**, making it easy for any web developer to use.

Mp2: The Platform-to-Infrastructure Interface

The **Mp2** reference point is used by the **MEC Platform Manager** to talk to the underlying **Virtualized Infrastructure Manager (VIM)**.

Functions over Mp2:

- Requesting the creation of a VM or container for an ME App.
- Querying for available compute and storage resources on the host.
- Configuring virtual networking for the application.

When the VIM is OpenStack, this interface would typically be the standard OpenStack Nova and Neutron APIs.

Mx2: The Platform-to-5G UPF Interface

The **Mx2** reference point is a critical interface between the **MEC Platform** and the **5G User Plane Function (UPF)**.

Functions over Mx2:

- The MEC Platform uses this interface to provide the UPF with the traffic filtering and forwarding rules needed for **local breakout**.
- For example, the platform tells the UPF: "For application XYZ, the traffic filter is protocol=UDP, dest_port=12345. If you see a packet that matches, forward it to this IP address (the ME App)."

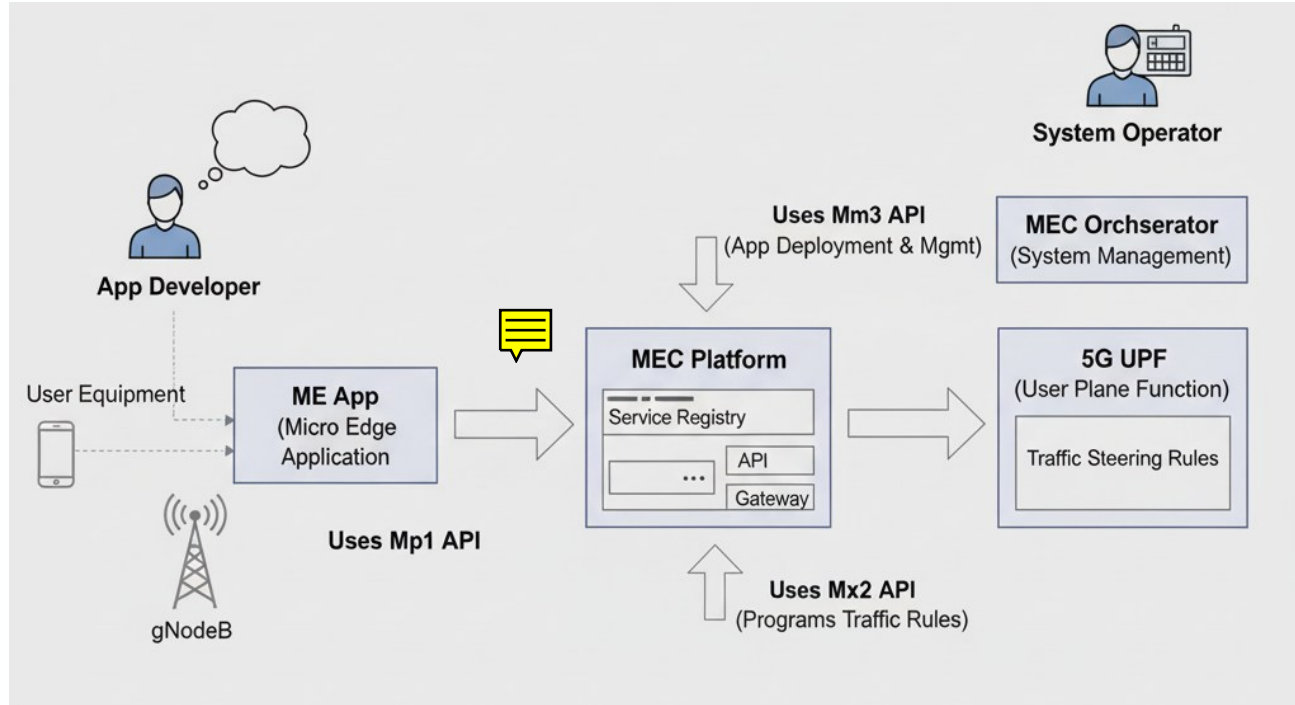
This interface is essential for dynamically controlling how user traffic is routed at the edge.

The "Mm" Series: Management Interfaces

The "Mm" reference points are a family of APIs used for all management and orchestration tasks.

- **Mm1**: Between the MEC Orchestrator and the OSS (the operator's top-level management system).
- **Mm3**: Between the MEC Orchestrator and the MEC Platform Manager (e.g., to instantiate an app).
- **Mm5**: Between the MEC Orchestrator and the VIM (e.g., for high-level resource queries).

Putting the Interfaces Together



Computational Offloading



Computation Offloading

We have explored the "what" and "where" of MEC architecture. Now we dive into the "how."

How do applications actually *use* the edge?

We will focus now on **computational offloading**, the core process where a mobile device sends a heavy task to the powerful MEC server for execution.

We will model this problem mathematically, walk through a detailed numerical example, and explore advanced concepts.

What is Computational Offloading?

The Problem: Mobile devices are fundamentally constrained by their limited battery life and processing power. Many desirable future applications (real-time AI, photorealistic AR) are too demanding to run locally.

The Solution: Offload the computationally intensive parts of the application from the resource-constrained device to a resource-rich server at the network edge.

Analogy: Think of your smartphone as a small, local workshop. It's great for simple tasks. But for a complex, heavy-duty job, it sends the blueprints to a massive, powerful factory (the MEC server) to get the job done quickly, receiving the finished product back.

The Fundamental Trade-Off: Local vs. Remote

The decision of whether to offload a task is not always obvious. It involves a critical trade-off between **computation** and **communication**.

Executing Locally:

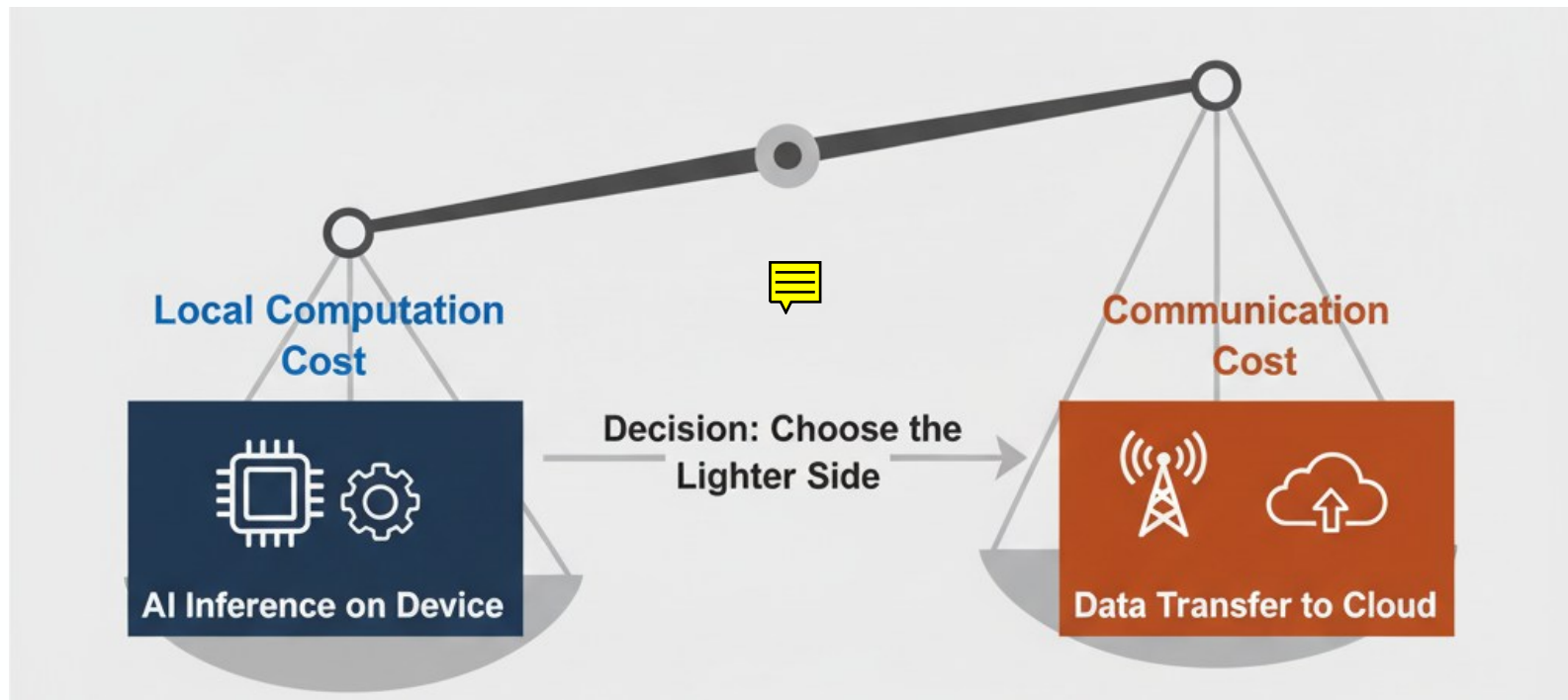


- **Cost:** High local CPU usage and high battery drain from processing.
- **Benefit:** Zero cost from using the network. There is no transmission time or energy cost.

Executing Remotely (Offloading):

- **Cost:** Must pay a price in time and energy to send the input data over the wireless network and receive the result.
- **Benefit:** The computation itself is much faster and consumes no *device* energy.

Local Computation vs Communication



Primary Goal: Latency Minimization

For interactive applications, the primary goal is to minimize the **end-to-end latency**, which is the total time from when the user initiates a task to when they see the result.

- **Total Latency (Local)** = $Time_to_compute_locally$
- **Total Latency (Offload)** = $Time_to_send_input + Time_to_compute_on_MEC + Time_to_receive_output$



The decision-maker must choose the option that results in the lowest total time. This is the critical goal for applications like AR/VR, cloud gaming, and real-time control.

Primary Goal: Energy Minimization

For battery-powered mobile devices, minimizing energy consumption is often just as important as minimizing latency. The goal is to maximize the device's operational lifetime.

- **Total Energy (Local)** = $Energy_to_compute_locally$
- **Total Energy (Offload)** = $Energy_to_send_input + Energy_to_receive_output +$
 $Energy_consumed_while_idle$

Note that the MEC server's computation energy is irrelevant to the device itself. The device aims to offload the task to save *its own* battery. This is the critical goal for background tasks or on devices with low battery.

Conflicting Goals

The lowest-latency option is not always the lowest-energy option.

Scenario: A user has a very fast but power-hungry 5G connection (e.g., mmWave).

- Offloading might be extremely fast, minimizing latency.
- However, the high transmit power required by the radio might consume more total energy than just computing the task locally with the CPU in a low-power state.

An intelligent offloading system must often choose which goal to prioritize, or find a solution that offers a good balance between the two. This decision might even be user-configurable ("Performance Mode" vs. "Battery Saver Mode").

Formalizing the Offloading Problem

To create an algorithm that can make an intelligent offloading decision, we need to move from concepts to mathematics.

We must create a formal model of the key components in the system:

1. The Application Task
2. The Mobile Device
3. The Network Link
4. The MEC Server

Modeling the Application Task

We can abstract any offloadable task, T , as a tuple of its key characteristics $T = \langle D_{in}, C, D_{out} \rangle$:

- **D_{in} (Input Data Size)**: The amount of data that must be sent to the server to execute the task. Measured in **bits**.
 - *Example: A 2 Megabyte image = 16×10^6 bits.*
- **C (Computational Complexity)**: The total number of CPU cycles required to complete the task. This is a measure of the "work" involved, independent of the processor speed. Measured in **cycles**.
 - *Example: An AI inference task might require 5 Giga-cycles.*
- **D_{out} (Output Data Size)**: The amount of data (the result) that must be sent back to the device. Measured in **bits**.
 - *Example: The result might be a simple text label = 1 kilobyte = 8000 bits.*

Modeling the Mobile Device



We model the device by its computation and communication capabilities:

- f_{local} (**Local CPU Speed**): The processing speed of the device's CPU. Measured in **cycles per second** (Hz).
 - *Example: A smartphone CPU might run at 2 Giga-cycles/sec (2 GHz).*
- P_{comp} (**Computation Power**): The power consumed by the CPU when it is actively processing. Measured in **Watts**.
- P_{tx} (**Transmit Power**): The power consumed by the device's radio when it is actively transmitting data. Measured in **Watts**. This often depends on the radio technology (**5G consumes more than Wi-Fi**).
- P_{idle} (**Idle Power**): The power consumed by the device when it is waiting for a response.

Modeling the Network

The wireless link between the device and the MEC host is a critical component. We model its key characteristics:

- B_{up} (**Uplink Bandwidth**): The achievable data rate from the device to the MEC host. Measured in **bits per second**.
- B_{down} (**Downlink Bandwidth**): The achievable data rate from the MEC host to the device. Measured in **bits per second**.

These values are **highly dynamic** and can be obtained from the MEC platform's Radio Network Information Service (RNIS).

Modeling the MEC Server

We model the edge server by its processing capability:

- f_{mec} (**MEC CPU Speed**): The processing speed of the CPU allocated to our task on the MEC server. Measured in **cycles per second** (Hz).
 - *Example: A powerful edge server might offer 20 Giga-cycles/sec (20 GHz) to a single task.*
- **Queuing Delay (T_q)**: The MEC server is a shared resource serving many users. If the server is busy, our task might have to wait in a queue before it gets processed. This waiting time is the **queuing delay**. For now, we will assume it is zero, but we will revisit this important factor later.

Offloading Strategies



Given our model, we can now define different strategies for how an application can be offloaded. The two main categories are:

1. **Binary Offloading**: The simplest model, where the entire task is treated as a monolithic, indivisible unit.
2. **Partial Offloading**: A more advanced model, where the task can be partitioned into sub-tasks, some of which can be offloaded while others run locally.

Strategy 1: Binary Offloading

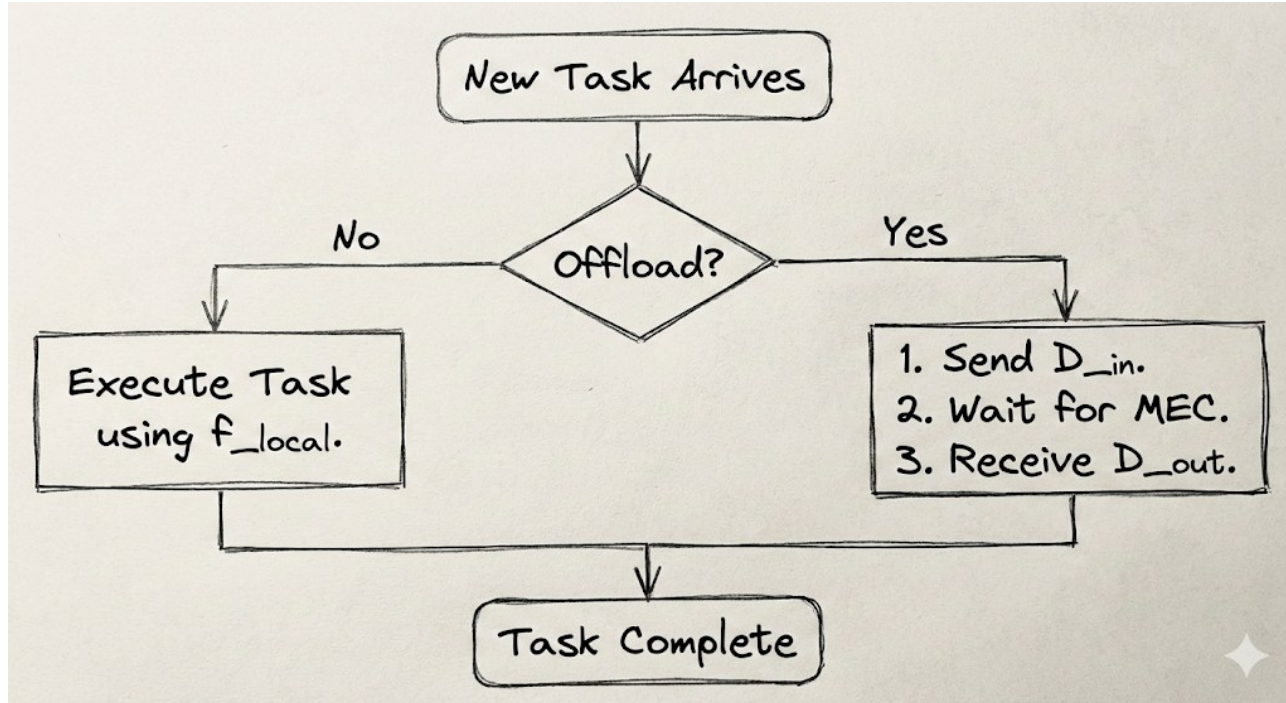
In this model, the decision is a simple binary choice:

- **Decision** $x = 0$: Execute the **entire task** locally on the mobile device.
- **Decision** $x = 1$: Offload the **entire task** to the MEC server.

The algorithm's job is to calculate the cost (latency or energy) for both $x = 0$ and $x = 1$, and then choose **the option with the lower cost.**

This is the model we will use for our main numerical example.

Offloading Decision Flowchart



Strategy 2: Partial Offloading

Many applications are not monolithic. They can be modeled as a **task graph**, a set of dependent sub-tasks.

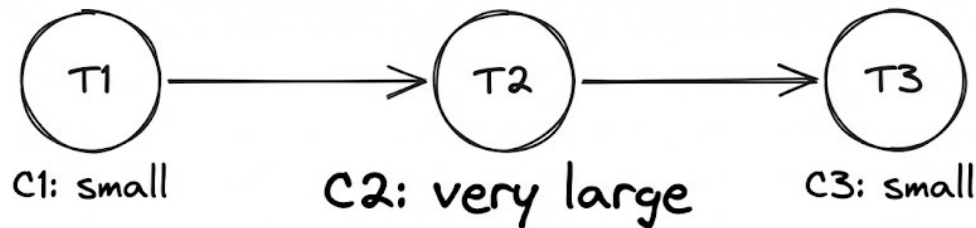
Example: An image processing application might have three steps:

- T1: Pre-process the image (e.g., resize).
- T2: Run a heavy AI object detection model.
- T3: Render the results onto the screen.

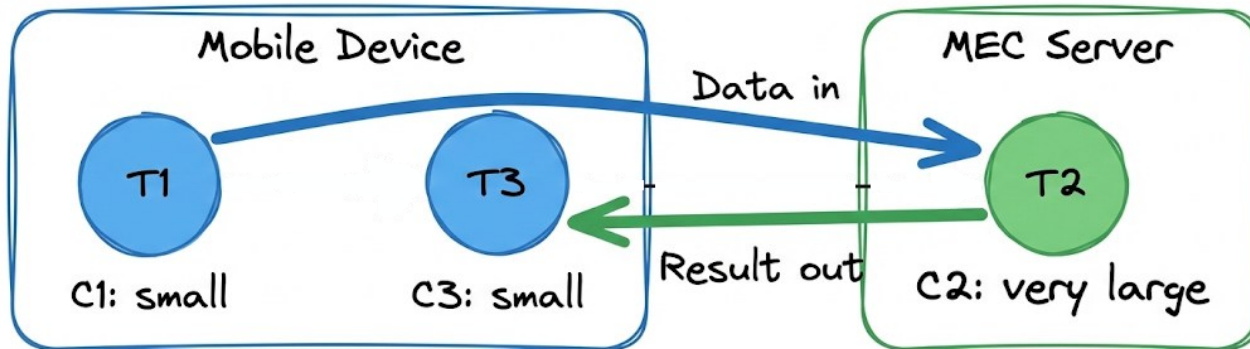
The Insight: It might be optimal to run T1 and T3 (which are lightweight or require local hardware) on the device, and **only offload the heavy T2 sub-task.**

This turns the decision problem from a simple binary choice into a much more complex graph partitioning problem.

Visualizing Partial Offloading



After Offloading Decision:



A Practical Example: The Offloading Decision

Let's apply the formal models we've just created to a realistic scenario.

Goal: We will walk through the step-by-step calculation to decide whether a mobile device should execute a **demanding Augmented Reality** task locally or offload it to a nearby MEC server.

We will analyze the decision from two perspectives: **minimizing latency** and **minimizing device energy consumption**.

Crucially, we will see how the **optimal decision changes dramatically** based on the quality of the network connection.

Step 1: Profile the Application Task

Our application is an AR-based object recognizer. The user points their phone camera at an object, and the app identifies it. The computationally intensive part is the AI model for image recognition.

We profile the task $T = \langle D_{in}, C, D_{out} \rangle$:

- **D_{in} (Input Data)**: A high-resolution camera image.
 - Size = **2 Megabytes (MB)**
 - In bits = $2 * 10^6 \text{ bytes} * 8 \text{ bits/byte} = \mathbf{16 \times 10^6 \text{ bits}}$.
- **C (Computational Complexity)**: The number of CPU cycles the AI model needs to process one image.
 - Complexity = **5 Giga-cycles = 5×10^9 cycles**.
- **D_{out} (Output Data)**: The result is just a text label with the object's name and confidence score.
 - Size = **1 Kilobyte (KB)**
 - In bits = $1 * 10^3 \text{ bytes} * 8 \text{ bits/byte} = \mathbf{8 \times 10^3 \text{ bits}}$.

Step 2: Profile the System Components

We define the parameters of our hardware and network.

Mobile Device:

- CPU Speed (f_{local}): **2 Giga-cycles/sec** (2 GHz).
- CPU Power (P_{comp}): **2.5 Watts** (when active).
- Transmit Power (P_{tx}): **1.5 Watts** (when radio is sending).
- Idle Power (P_{idle}): **0.1 Watts** (when waiting for a response).

MEC Server:

- CPU Speed (f_{mec}): **25 Giga-cycles/sec** (a powerful server-class CPU).

Network States: We will analyze two different scenarios.

- **Scenario A: Good 5G Coverage:** $B_{up} = B_{down} = \mathbf{100\ Mbps}$ (10^8 bits/sec).
- **Scenario B: Poor 4G Coverage:** $B_{up} = B_{down} = \mathbf{5\ Mbps}$ ($5 * 10^6$ bits/sec).

Option 1: Calculating Local Execution Time



First, let's calculate the cost of running the entire task on the smartphone. This result is the same regardless of network conditions.

Formula:

$$\circ T_{local} = T_{comp_{local}} = C/f_{local} = 2.5 \text{ seconds}$$

Calculation:

$$\circ T_{local} = T_{comp_{local}} = (5 \cdot 10^9 \text{ cycles}) / (2 \cdot 10^9 \text{ cycles/sec}) = 2.5 \text{ seconds}$$

Conclusion: It takes **2.5 seconds** to identify the object when running on the phone. For an interactive AR application, this is unacceptably slow.

Option 1: Calculating Local Execution Energy

Now, let's calculate the energy drained from the device's battery for this local computation.

Formula:

$$\circ E_{local} = P_{comp} \cdot T_{comp_{local}}$$

Calculation:

$$\circ E_{local} = 2.5 \text{ Watts} \cdot 2.5 \text{ seconds} = 6.25 \text{ Joules}$$

Conclusion: Executing the task locally consumes **6.25 Joules** of energy. This is our baseline for comparison.

Option 2 (Good 5G): Upload Time & Energy

Now we analyze the offloading option, starting with the cost to send the 2 MB image to the MEC over the fast 100 Mbps 5G link.

Upload Time (T_{up}):

- $T_{up} = D_{in}/B_{up} = (16 \cdot 10^6 \text{ bits})/(10^8 \text{ bits/sec}) = 0.16 \text{ seconds}$



Upload Energy (E_{up}):

- $E_{up} = P_{tx} \cdot T_{up} = 1.5 \text{ Watts} \cdot 0.16 \text{ seconds} = 0.24 \text{ Joules}$



Option 2 (Good 5G): MEC Execution Time & Idle Energy

Next is the time it takes the powerful MEC server to perform the computation. 

Note: For this example, we will assume the queuing delay (T_q) is zero. In a real system, this would need to be measured or estimated.

Formula:

$$\circ T_{mec_{comp}} = C / f_{mec}$$

Calculation:

$$\circ T_{mec_{comp}} = (5 \cdot 10^9 \text{ cycles}) / (25 \cdot 10^9 \text{ cycles/sec}) = 0.2 \text{ seconds}$$

During the MEC computation time, the device's radio is idle but still consuming some power.

Energy Consumed While Idle (E_{idle}):

$$\circ E_{idle} = P_{idle} \cdot T_{mec_{comp}} = 0.1 \text{ Watts} \cdot 0.2 \text{ seconds} = 0.02 \text{ Joules.}$$

Option 2 (Good 5G): Download Time & Energy

Finally, we calculate the cost to receive the small 1 KB result.

Download Time (T_{down}):

$$\circ T_{down} = D_{out}/B_{down} = (8 \cdot 10^3 \text{ bits}) / (10^8 \text{ bits/sec}) = 0.00008 \text{ seconds (negligible)}.$$

Download Energy (E_{down}):

$$\circ E_{down} = P_{rx} \cdot T_{down}$$



The energy to receive the tiny result is negligible.

Option 2 (Good 5G): Total Offloading Cost

Now we sum the parts to get the total cost of offloading.

Total Time (T_{offload}):

$$\circ T_{\text{offload}} = T_{\text{up}} + T_{\text{mec}_{\text{comp}}} + T_{\text{down}} = 0.16 \text{ s} + 0.2 \text{ s} + 0.00008 \text{ s} = 0.36008 \text{ seconds}$$

Total Device Energy (E_{offload}):

$$\circ E_{\text{offload}} = E_{\text{up}} + E_{\text{idle}} + E_{\text{down}} = 0.24 \text{ J} + 0.02 \text{ J} + (\text{negligible}) = 0.26 \text{ Joules}$$

The Decision: Scenario A (Good 5G)

Let's compare our results in a clear table.



Metric	Local Execution	MEC Offloading	Winner
Latency	2.5 seconds	0.36 seconds	MEC Offloading (6.9x faster)
Device Energy	6.25 Joules	0.26 Joules	MEC Offloading (24x more efficient)

Conclusion: In a good 5G environment, the decision is obvious and overwhelming. **Offloading the task** is massively better in every way. It provides a real-time user experience while saving a huge amount of battery.

What if the Network Changes?

Now, imagine the user walks into an area with poor signal strength. Their connection drops from a fast 100 Mbps 5G to a slow, congested 5 Mbps 4G link.

All other parameters (the task, the device, the MEC server) remain the same.

The cost of local execution is unchanged (it doesn't depend on the network).

Let's re-calculate the cost of offloading.

Option 2 (Poor 4G): Re-calculating the Upload Cost

The only thing that changes is the bandwidth B. Let's re-calculate the upload time and energy.

Upload Time (T_{up}):

$$\circ T_{up} = D_{in}/B_{up} = (16 \cdot 10^6 \text{ bits})/(5 \cdot 10^6 \text{ bits/sec}) = 3.2 \text{ seconds}$$

Upload Energy (E_{up}):

$$\circ E_{up} = P_{tx} \cdot T_{up} = 1.5 \text{ Watts} \cdot 3.2 \text{ seconds} = 4.8 \text{ Joules}$$

Immediate Insight: Just sending the data now takes longer than the entire local computation time!

Option 2 (Poor 4G): Total Offloading Cost

Now we re-calculate the total cost with the new, much larger upload time.

Total Time ($T_{offload}$):

$$\circ T_{offload} = T_{up} + T_{mec_{comp}} + T_{down} = 3.2 s + 0.2 s + (negligible) = 3.4 seconds$$

Total Device Energy ($E_{offload}$):

$$\circ E_{offload} = E_{up} + E_{idle} + E_{down} = 4.8 J + 0.02 J + (negligible) = 4.82 Joules$$

The Decision: Scenario B (Poor 4G)

Let's look at the new comparison table.

Metric	Local Execution	MEC Offloading	Winner
Latency	2.5 seconds	3.4 seconds	Local Execution
Device Energy	6.25 Joules	4.82 Joules	MEC Offloading

Conclusion: The optimal decision has **completely flipped**.

To **minimize latency**, the device must now **execute the task locally**. The network is too slow to make offloading worthwhile.

Interestingly, offloading still saves a small amount of energy. This shows how the goals can conflict. For an interactive app, latency is king, **so the choice is clear: Do not offload.**

Key Insights from the Example

The optimal offloading decision is not static; it is highly **dynamic** and **context-dependent**.

The **ratio of computation to communication** is key. Offloading is beneficial for tasks that are very computationally heavy but have relatively small input/output data (high C/D_{in} ratio).

Network conditions are the dominant factor. A change in available bandwidth can completely reverse the optimal decision.

This is why an intelligent MEC application needs access to the **Radio Network Information Service (RNIS)** to get real-time network measurements before making an offloading decision.

Advanced Topics in Computational Offloading

Our numerical example made some simplifying assumptions (like a dedicated server and a stationary user). In the real world, the problem is even more complex and dynamic.

Let's explore some of these advanced challenges and the research being done to solve them:

- The Multi-User Dimension: Sharing the Edge
- The Mobility Problem: Offloading on the Move
- The AI-Driven Solution: Learning to Offload

Advanced Topic: The Multi-User Dimension

A MEC server is not dedicated to a single user; it's a **shared resource** serving hundreds or thousands of users simultaneously within a cell.

This leads to **resource contention**. Multiple users might try to offload tasks at the same time, competing for:

- CPU cycles on the MEC server.
- Radio resources (bandwidth) in the cell.

An offloading decision made by one user can negatively impact the performance for another. For example, if you offload a large task, you might consume bandwidth that a nearby user also needed.

Modeling Queuing Delay on the MEC

Because the server is shared, our assumption that queuing delay (T_q) is zero is not realistic.

When your offloaded task arrives at the MEC server, it might have to wait in a queue if the server's CPUs are already busy processing tasks from other users.

This can be modeled using **Queuing Theory**. For example, a simple M/M/1 model can predict the average waiting time based on the task arrival rate and the server's processing rate.

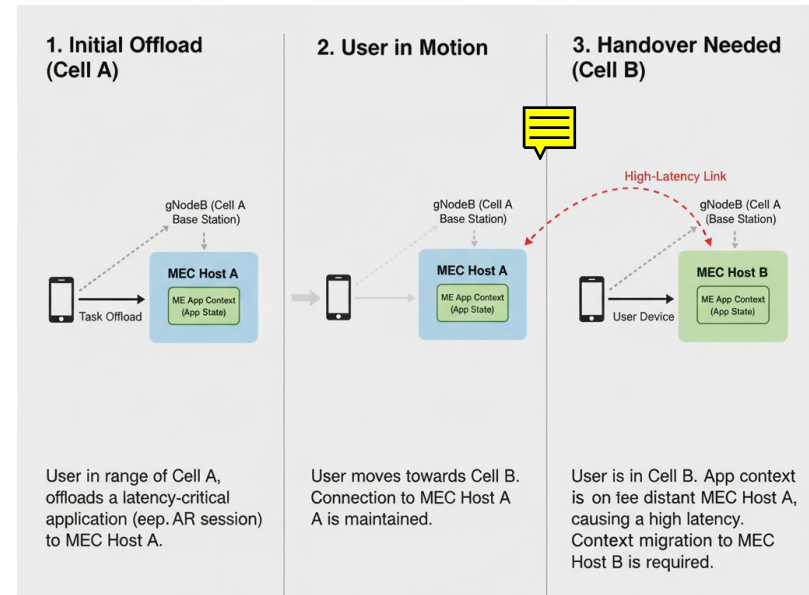
The Impact: $T_{offload} = T_{up} + T_q + T_{mec_{comp}} + T_{down}$. A high queuing delay during peak hours could make offloading much slower, potentially reversing the optimal decision. The MEC orchestrator needs to manage this load carefully.

The Mobility Problem: Offloading on the Move

Users are, by definition, **mobile**. They move between different cell sites.

The Critical Question: What happens if a user starts offloading a long-running task to the MEC server at Cell Site A, but before the result is sent back, they move and connect to Cell Site B?

The Problem: The connection to the original MEC server at Site A is now lost or has very high latency. The user's application state is "stranded" on the old server, and the user experiences a service interruption.



Solutions for Mobility: Service Migration

The MEC Orchestrator (MEAO) is responsible for handling mobility to ensure service continuity. It has two main strategies:

1. **Service Re-instantiation:** The simple approach. When the user arrives at Cell B, the orchestrator terminates the old app instance at MEC A and starts a fresh new one at MEC B.



- **Pro:** Simple to implement.
- **Con:** All application state is lost. This is unacceptable for many applications (like a game).

2. **Live Service Migration:** The advanced approach. The orchestrator proactively copies the entire state of the running application (memory, storage) from MEC A to MEC B just before the user completes the handover.



- **Pro:** Provides a seamless user experience with no loss of state.
- **Con:** Technically very complex and consumes significant network resources to transfer the state.

AI-Driven Offloading Decisions

As we've seen, the optimal offloading decision is a complex, multi-variable problem that depends on:

- The application's profile.
- The device's state (battery level).
- The dynamic network conditions (bandwidth, latency).
- The MEC server's state (current load, queuing delay).
- The user's mobility pattern.

Calculating all this with a perfect mathematical model in real-time is nearly impossible. This makes it a perfect problem for **Artificial Intelligence and Machine Learning**.

Reinforcement Learning for Offloading

A promising approach is to use **Reinforcement Learning (RL)** to create an intelligent offloading agent on the device or in the network.

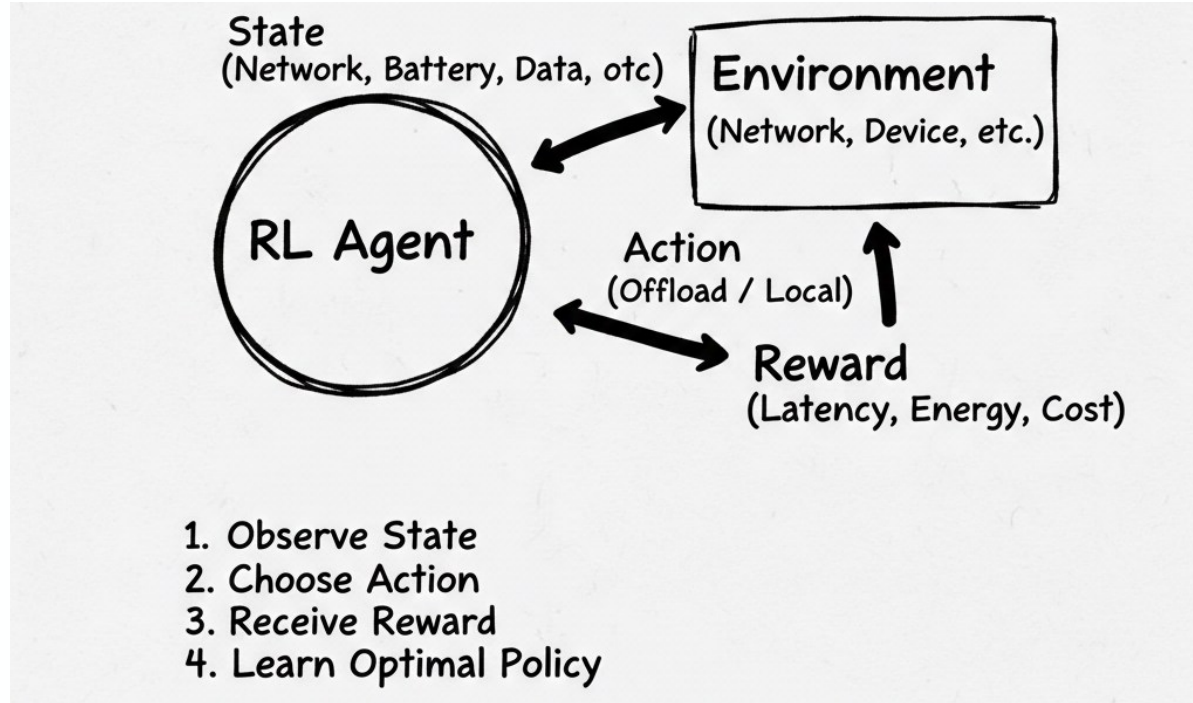
How it works:

- The **RL agent** observes the current **state** (network conditions, battery level, etc.).
- It takes an **action** (decide to offload or run locally).
- It receives a **reward** based on the outcome (e.g., a high reward for low latency, a penalty for high energy consumption).

Over time, through trial and error, the RL agent **learns** a complex policy that maps states to the optimal action, without needing a perfect mathematical model of the world.

Relevance: This is a core research area for students in the **Advanced Computing** curriculum.

Reinforcement Learning for Offloading



Use Cases & The Ecosystem



Use Cases & The MEC Ecosystem

We have now mastered the architecture and core mechanisms of MEC. In this final part, we will connect this technology to tangible, real-world applications.

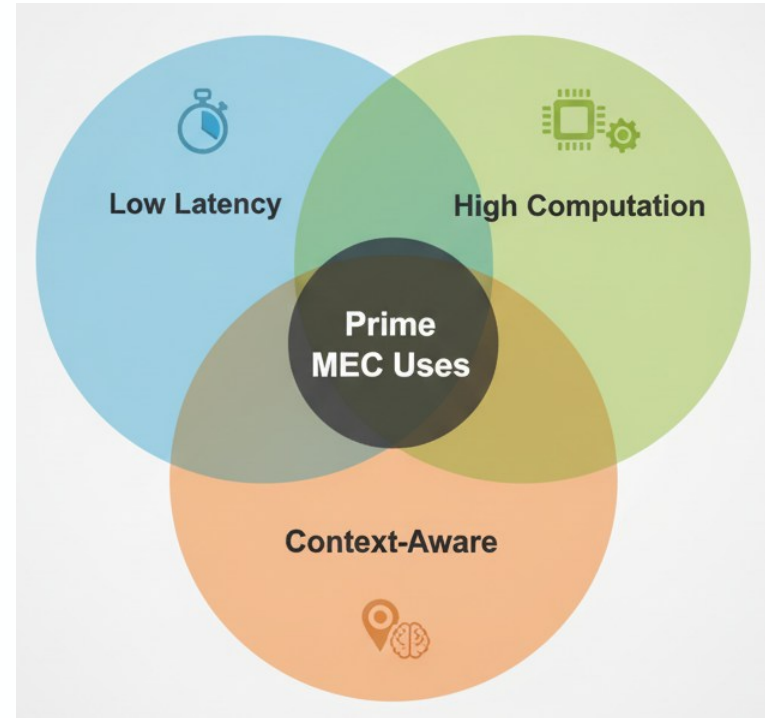
We will explore in detail:

- **Key Use Cases:** A technical deep dive into the applications that are enabled by MEC, with specific links to your different curricula.
- **The Business & Technical Ecosystem:** Who are the key players, what are their roles, and what are the open technical challenges?
- **Future Outlook:** What's next for MEC as we move towards 6G?

Who Benefits from MEC?

MEC is not a universal solution; it is a specialized tool for a specific class of applications. The applications that benefit the most are those that are simultaneously:

- **Interactive:** Requiring very low latency for a good user experience.
- **Computationally Intensive:** Requiring more processing power than a mobile device can offer.
- **Context-Aware:** Able to leverage real-time information about the user's location or network conditions.



Standardization Bodies

3GPP:

- Focus Areas: Primarily focuses on radio and core network technologies.
- Edge Computing Integration: From Release 15 onwards, 3GPP has started integrating edge computing into the 5G system architecture.
- Detailed Support: Further detailed support for edge computing is being defined from Release 17 onwards.

ETSI ISG MEC:

- Dedicated Group: A standard group specifically created to define edge computing technology.
- IT Perspective: Defines how edge computing is realized from an IT perspective, in an access-agnostic manner.
- Cloud Technologies: Specifies the set of cloud technologies to be implemented by MEC providers and application/content providers.

Industry Collaboration

Wide Applicability: MEC technology is applicable to a variety of 5G use-cases and service scenarios, such as automotive and industrial automation.

Specialized Associations: Various industry associations (e.g., 5GAA) are emerging to study each vertical market segment in detail and create consensus among stakeholders.

Example - 5GAA:

- **Automotive Focus:** 5GAA (5G Automotive Association) connects telecom industry and vehicle manufacturers to develop solutions for future mobility and transportation services.
- **Stakeholder Collaboration:** Involves telecommunication companies (providing connectivity and networking systems) and automotive players (working on vehicle platforms and solutions).

Industry Collaboration

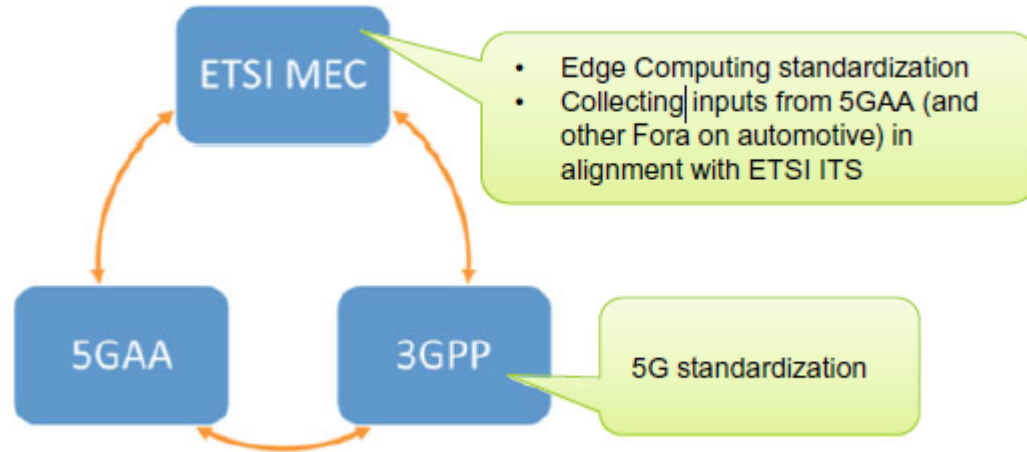
Challenges and Synergies:

- Harmonizing Views: The main challenge is to harmonize the perspectives of telecom and automotive stakeholders to produce comprehensive technical requirements and solutions.
- Interoperability: Ensuring interoperability among different players is crucial, especially in complex ecosystems like automotive.
- Standardization Impact: 5GAA, as an industry group, does not publish standards but influences standard bodies by building consensus and producing solutions for standard contributions.

Conclusion:

- Complementary Work: ETSI MEC and 3GPP work is complementary and synergic, with different focuses but a unified goal.
- Coherent Standards: The aim is to ensure coherent and compatible standards for smooth deployment of MEC technologies in 5G systems.

Example of synergy between ETSI MEC, 3GPP standard and industry group (5GAA, 5G Automotive Association)



Use Case: Augmented & Virtual Reality (AR/VR)

The Challenge: To create a truly immersive and believable experience, AR/VR systems must generate and display photorealistic images that react to the user's head movements almost instantly. Any noticeable lag between a head movement and the visual update causes nausea and breaks the illusion of presence.

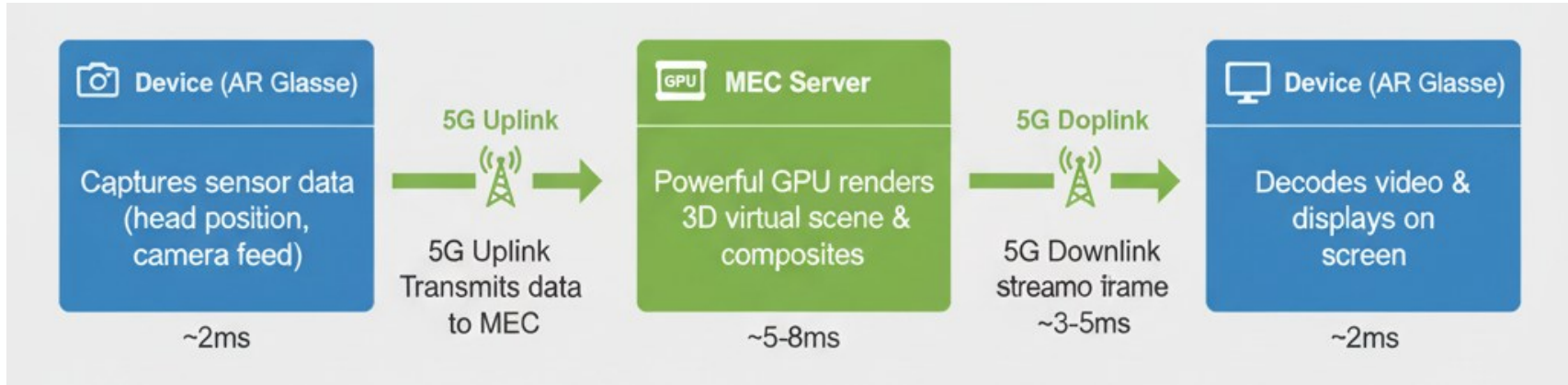
The Requirement: This "motion-to-photon" latency must be **less than 20 milliseconds**.

The Problem: Untethered, all-day wearable AR glasses will never have the battery or processing power to render these complex scenes locally.

Relevance: Core use case for **Multimedia Systems**.

AR/VR: The MEC-Enabled Split Rendering Pipeline

MEC is the only viable solution for high-quality, untethered AR/VR. The application logic is "split" between the device and the edge.



Total Round-Trip Latency: Summing the steps gives a total of ~15-25ms, which meets the critical requirement for interactive AR experiences.

AR/VR: The Role of MEC in 6DoF Tracking

6DoF (Six Degrees of Freedom) refers to the ability to track a user's position (x, y, z) and orientation (pitch, yaw, roll) in 3D space. This is essential for allowing users to walk around in an AR/VR environment.

The Challenge: This requires complex sensor fusion and Simultaneous Localization and Mapping (SLAM) algorithms.

The MEC Solution: The device can offload the heavy SLAM computations to the MEC server. The MEC can also host a persistent, high-detail 3D map of the environment (a "digital twin" of the area), which multiple users can share and interact with, enabling collaborative AR experiences.

Use Case: Real-Time Video Analytics

The Goal: To use AI/ML to automatically analyze live video streams and extract valuable insights or trigger immediate actions.

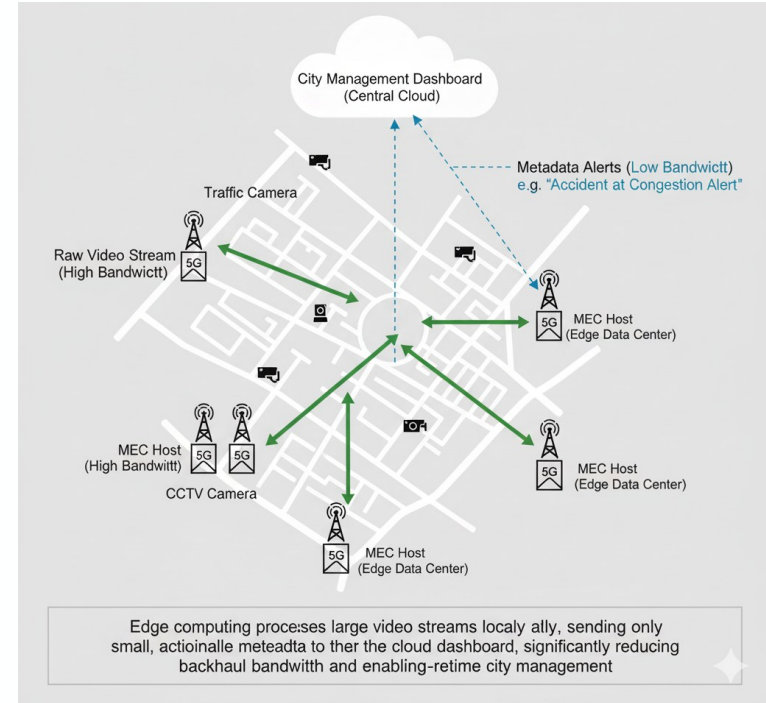
The Spectrum of Applications:

- Public Safety: Detecting traffic accidents, unauthorized intrusions, or overcrowding in public spaces.
- Retail Analytics: Analyzing customer footfall, dwell times in front of displays, and queue lengths to optimize store layout.
- Industrial Quality Control: Visually inspecting products on an assembly line for defects.

Relevance: Core use case for **Big Data/Distributed Systems** and **Advanced Computing**.

Video Analytics: A Smart City Deployment

A MEC-based architecture is the only scalable way to deploy video analytics across a city.



Video Analytics: The Interdisciplinary Challenge

This single use case highlights the need for skills from multiple curricula:

- **Big Data / Distributed Systems:** You design the distributed data pipeline. How is video ingested? How is metadata stored, indexed, and queried? How do you manage a fleet of distributed analytics applications?
- **Advanced Computing:** You develop and optimize the AI models. Which neural network architecture is best for real-time object detection on an edge GPU? How do you deploy and update these models without service interruption?
- **Multimedia Systems:** You handle the video itself. Which encoding (e.g., H.265) and transport protocol provides the best quality and lowest latency from the camera to the MEC?

Use Case: Connected Cars (V2X)

The Vision: To create a cooperative transportation system where vehicles, infrastructure, and pedestrians share information to prevent accidents and improve traffic flow.

The Challenge: Safety-critical information must be shared and processed with extremely high reliability and ultra-low latency. A warning that arrives 100 milliseconds too late is useless.

Relevance: Core use case for **CSN (URLLC)** and **Advanced Computing**.

V2X: The Role of MEC as an Information Broker

A key V2X concept is **collective perception**, where entities share sensor data to build a richer view of the world than any single entity could have on its own.

The Problem: If every car and sensor broadcasted all its data to everyone else, the radio channels would be instantly overwhelmed (a "broadcast storm").

The MEC Solution: The MEC server acts as an intelligent, local **information broker**.

- Vehicles and sensors send their data (e.g., CAM - Cooperative Awareness Messages) to the local MEC.
- The MEC aggregates, filters, and fuses this information.
- It then sends only the most relevant and critical warnings (e.g., DENM - Decentralized Environmental Notification Messages) to the specific vehicles that need them.

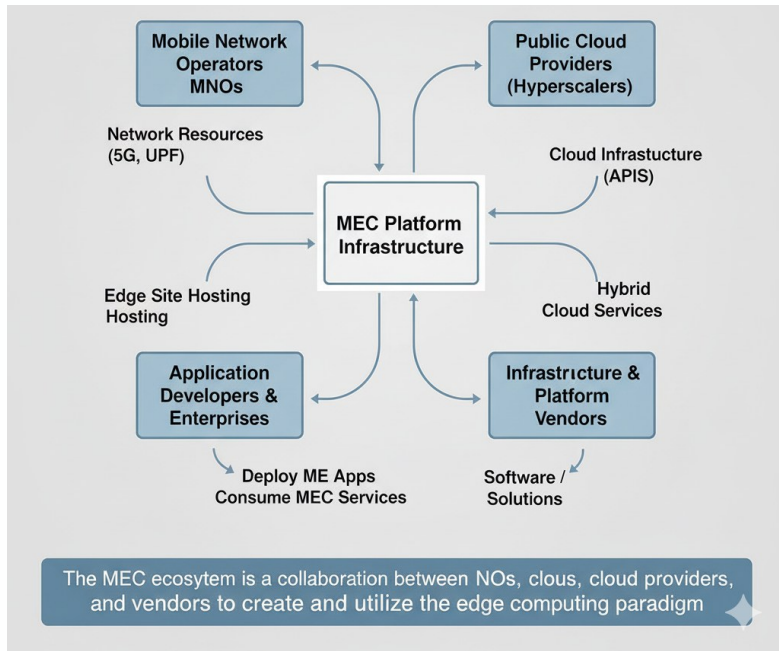
V2X: Curricula Relevance

Computing Systems & Networks: You design the URLLC network slice and protocols to guarantee that a V2X safety message is delivered in under 5ms with 99.999% reliability.

Advanced Computing: You design the fusion and decision-making algorithms on the MEC. How do you process sensor data from 50 different sources to accurately predict a potential collision?

Big Data / Distributed Systems: You design the data models and platforms for handling this massive, real-time, geospatial data stream.

The MEC Ecosystem: Who Are the Players?



Creating and deploying MEC services requires a complex partnership between different types of companies.

The Mobile Network Operator's (MNO) Role

The MNO is a crucial player because they own the most valuable and difficult-to-replicate asset: the **"edge real estate"**.

They own the cell towers, the spectrum licenses, and the physical locations where MEC hosts can be deployed in close proximity to users.

Their Business Model: To monetize their 5G network investment by selling low-latency access and network APIs to application developers and cloud providers.

The Public Cloud Provider's Role

The cloud providers have the massive developer ecosystem and the easy-to-use tools (APIs, orchestration platforms) that developers love.

Their Strategy: They don't want to build their own 5G networks. Instead, they **partner with MNOs** to extend their cloud platform to the operator's edge.

Real-World Examples:

- **AWS Wavelength:** Deploys AWS compute and storage services at the edge of MNOs' 5G networks.
- **Microsoft Azure Edge Zones:** A similar offering from Microsoft.

This allows a developer to use the same familiar AWS or Azure APIs to deploy parts of their application in the central cloud and other parts at the mobile edge.

The Application Developer's Perspective

For MEC to succeed, it must be easy for the millions of application developers to use.

What they need:

- **Simple, Abstracted APIs:** They don't want to learn complex telecom protocols. They want simple REST APIs for location, bandwidth, etc.
- **Familiar Tools:** The ability to package their apps in containers and manage them with standard tools like Kubernetes.
- **A Clear Business Model:** A simple way to pay for the edge resources they consume and a clear way to make money from their low-latency services.

Open Challenges

The "Killer App": While AR and V2X are promising, the single "killer application" that will drive massive, widespread MEC adoption has not yet fully emerged.

Interoperability: Ensuring that an application deployed via AWS Wavelength on TIM's network in Italy can seamlessly interact with one deployed on Verizon's network in the US is a major challenge.

Monetization: Who pays for what? Does the user pay their MNO for a "low latency" subscription? Does the app developer pay the MNO? Does the cloud provider pay the MNO? These value chains are still being defined.

Part 1 Summary: MEC

What it is: A standardized architecture for deploying cloud-native applications at the edge of the mobile network.

Why it matters: It is the only way to meet the **ultra-low latency** demands of future applications like AR/VR, V2X, and real-time AI.

How it works: Through **computational offloading**, where devices send heavy tasks to the powerful, nearby MEC server. The decision to offload is dynamic and context-dependent.

How it's built: The ETSI framework defines the key components (Host, Platform, Orchestrator) and the open APIs (Mp1, Mx2) that enable an interoperable ecosystem.

Who's involved: It requires a deep partnership between MNOs, cloud providers, and application developers.

The Future of MEC: Towards 6G

As we move towards 6G, the role of the edge will become even more critical.

"Network as a Sensor": The 6G vision of Integrated Sensing and Communication (ISAC) will rely on MEC. The vast amount of raw sensing data collected by the network will have to be processed at the edge.

Holographic Telepresence: The massive computational load required to render real-time holograms will be processed on next-generation, AI-accelerated MEC hosts.

Pervasive AI: In 6G, AI will not just be an application running at the edge; the edge platform itself will be AI-native, with intelligent, autonomous orchestration.

The Future of MEC: AI-Native Platforms

The MEC orchestrator of the future will be driven by AI.

Predictive Placement: Using ML to predict user mobility and proactively instantiate applications on the *next* MEC host before the user even arrives.

Intelligent Resource Management: Using reinforcement learning to dynamically allocate radio and compute resources to maximize overall QoE for all users.

Autonomous Healing and Optimization: The platform will automatically detect, diagnose, and fix problems without human intervention.

Final Takeaways on MEC

Mobile Edge Computing is a fundamental paradigm shift that **blurs the line between the network and the computer.**

It transforms the mobile network from a simple connectivity pipe into a **distributed application platform.**

Mastering the principles of MEC—from its architecture to its core offloading mechanisms—is essential for any engineer aiming to build the next generation of intelligent, real-time, context-aware applications and services.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.1: Mobile Edge Computing (MEC)

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

daniele.tarchi@unifi.it